RedFuser: An Automatic Operator Fusion Framework for Cascaded Reductions on AI Accelerators

Xinsheng Tang*
Alibaba Cloud Computing
Shanghai, China

Yangcheng Li*
Alibaba Cloud Computing
Shanghai, China

Nan Wang*
Alibaba Cloud Computing
Shanghai, China

Zhiyi Shu
Alibaba Cloud Computing
Shanghai, China

Xingyu Ling
Alibaba Cloud Computing
Shanghai, China

Junna Xing
Alibaba Cloud Computing
Shanghai, China

Peng Zhou
Alibaba Cloud Computing
Sunnyvale, USA

Qiang Liu
Alibaba Cloud Computing
Shenzhen, China

Abstract

Operator fusion, as a key performance optimization technique in the deployment of AI models, significantly improves execution efficiency and has been widely adopted in modern AI compilers. However, for cascaded reduction operations involving multiple loops with inter-loop data dependencies, such as the safe softmax followed by GEMM within attention mechanisms, existing compilers lack effective automated fusion and kernel generation capabilities. Although some works have addressed specific instances through handcrafted fusion strategies, their solutions are limited in generality and difficult to extend to other similar structures. Given the prevalence of such computational patterns in deep learning models, there remains significant untapped potential in achieving general and automated fusion optimization.

In this paper, we present a formal theoretical methodology for analyzing cascaded reductions which can fuse them into a single loop and introduce an incremental computation form. Based on this methodology, we design **Red**uction **Fuser** (RedFuser), a framework that automatically identifies supported cascaded reduction patterns and generates optimized fused kernels. Experiments show that RedFuser successfully fuses diverse workloads, achieving up to 2× to 5× speedup over state-of-the-art AI compilers and matching the performance of highly optimized hand-written kernels.

*All three authors contributed equally to this research.



This work is licensed under a Creative Commons Attribution 4.0 International License.

ASPLOS '26, Pittsburgh, PA, USA.
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2359-9/2026/03
https://doi.org/10.1145/3779212.3790209

CCS Concepts: • **Computer systems organization** → *Parallel architectures*; • **Computing methodologies** → **Parallel computing methodologies**; *Artificial intelligence*; • **Software and its engineering** → **Compilers**.

Keywords: Automatic Operator Fusion, Cascaded Reductions, GPUs

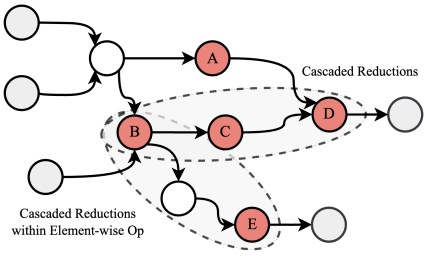
ACM Reference Format:
Xinsheng Tang, Yangcheng Li, Nan Wang, Zhiyi Shu, Xingyu Ling, Junna Xing, Peng Zhou, and Qiang Liu. 2026. RedFuser: An Automatic Operator Fusion Framework for Cascaded Reductions on AI Accelerators. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '26), March 21–26, 2026, Pittsburgh, PA, USA.* ACM, New York, NY, USA, 23 pages. https://doi.org/10.1145/3779212.3790209

1 Introduction

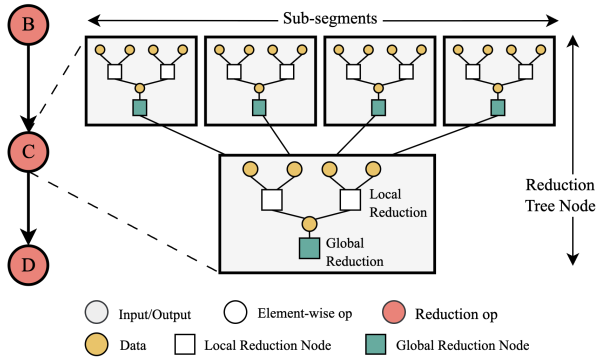
In recent years, artificial intelligence (AI) models have demonstrated remarkable advances in various technical domains [9, 14, 26, 27, 31, 33, 48], with state-of-the-art models [18, 19, 30, 34, 38] consistently achieving benchmark performance that approaches or exceeds human-level baselines. However, these technological achievements have introduced significant scaling challenges. Modern cut-ting-edge large language models (LLMs) now exceed trillion parameters (e.g., Kimi K2 [34] has 1 trillion parameters), greatly increasing computational cost. To meet the extreme demands for cost and speed, enhancing the efficiency of model training and inference becomes critically important.

Reductions are fundamental computational patterns in AI workloads that derive a single value from an array through loop-based mathematical operations, such as summation and maximum. As shown in Figure 1, a common parallel strategy for reductions on AI accelerators is the **reduction tree** structure. In this approach, the input sequence is partitioned into sub-segments for local computation and caching, followed by global reductions to aggregate results into a final output. Notably, the general matrix multiplication (GEMM)

operation, which multiples an M × K matrix A with an K × N matrix B, producing an M × N matrix C, can be expressed as a triple nested loop consisting of M × N independent reduction operations along the K-axis, making it fundamentally a reduction operation.



(a) Cascaded reduction patterns



(b) The chain of reduction trees

Figure 1. The reduction tree structure in cascaded reductions

In deep learning, sequences of reduction operations with data dependencies, referred to as **cascaded reductions**, are prevalent computational patterns. Examples include attention mechanisms and normalization layers. Cascaded reductions involving multiple loops naturally form chains of reduction trees. Unlike inherently parallelizable operations such as element-wise computations, cascaded reductions exhibit a critical computational characteristic – **data dependency**: each subsequent reduction requires the output of its predecessor, specifically, the root node of the prior reduction tree. This dependency introduces two key performance bottlenecks:

- **Redundant Memory Access**: Each reduction tree incurs independent memory accesses for its input data.
- **Limited Parallelism**: Reduction trees cannot be executed in parallel, nor can memory accesses be overlapped with reduction computations to hide latency, significantly limiting overall performance.

Moreover, each level of a reduction tree must cache the complete result from the previous level, increasing cache pressure and constraining the achievable reduction length per level.

To address these challenges, **operation fusion** [11, 20, 28, 29] is a widely adopted optimization technique that merges multiple consecutive operators into a single operator. However, existing AI compilers [1, 3, 24, 32, 41, 42, 46, 47, 49] exhibit critical limitations in optimizing cascaded reductions through operator fusion. Specifically, these approaches lack the capability to perform detailed analysis, leading to suboptimal fusion strategies. While some fine-grained manual optimization techniques [5, 6, 16, 39, 43] have addressed specific instances, they require careful case-by-case derivation and exhibit limited universality across different patterns.

To overcome these limitations, our work makes the following key contributions:

- **Cascaded Reductions Fusion Methodology**: We propose a formal methodology that fuses cascaded reductions into a single reduction, thereby eliminating redundant data loads and enabling parallel execution of dependent reductions. To mitigate the constraints of cache capacity at each level on the corresponding reduction length, we introduce an incremental computation scheme that updates partial results progressively. The methodology is designed to be general and compatible with the compilation stacks of modern AI compilers.
- **Lightweight Operator Fusion Framework**: We implement our methodology in **RedFuser**, a two-stage lightweight operator fusion framework that systematically optimizes cascaded reductions. First, it employs a symbolic deduction engine to analyze the fusibility of cascaded reductions, and derive the fused reduction expressions and corresponding incremental computation forms. Second, it performs hardware-aware code generation by lowering the fused expression into a tile-based representation and generating high-performance kernels.
- **Comparative Performance Evaluation of Various Workloads**: We evaluate RedFuser on modern GPU architectures (NVIDIA A10 and H800) across four representative workloads: Multi-Head Attention (MHA), Multi-Latent Attetion (MLA), MoE routing, and FP8 Quant + GEMM. Results show that RedFuser achieves 2× to 5× speedup over state-of-the-art AI compilers (e.g., TVM[3], Dynamo[1]), and its generated kernels match or exceed the performance of expert-optimized implementations, achieving state-of-the-art (SOTA) results.

2 Background and Motivation

2.1 Parallel Reduction on GPUs

Modern Graphics Processing Units (GPUs) have emerged as the cornerstone of accelerating compute-intensive applications, due to their massive parallelism and high memory bandwidth. At the core of GPU computing lies a hierarchical programming model [25] that organizes parallel execution into three levels: threads, thread blocks, and grids. Each thread possesses its own private local memory and executes

instructions independently. Threads are grouped into thread blocks, within which they can cooperate efficiently through a shared memory space, enabling fine-grained data reuse and synchronization. Direct synchronization or communication mechanisms between blocks are absent in the current programming model, unless via multiple kernel launches or features such as cooperative groups. All blocks collectively form a grid that executes a single kernel function. The memory hierarchy includes per-thread local memory, block-scoped shared memory, and globally accessible global memory, providing a balanced trade-off between capacity and access latency. Programming frameworks such as CUDA enable developers to implement parallel GPU algorithms through techniques such as tiling, data reuse, and explicit memory management.

Parallel reduction on GPUs is a well-studied problem. The seminal work by Harris [13] established a systematic methodology for efficient GPU-based reduction. The input data are first partitioned into multiple segments, with each thread block responsible for reducing one segment locally. Specifically, each thread loads its assigned elements from global memory into shared memory. Within the block, a sequence of reduction steps is performed using an interleaved addressing pattern: in each iteration, pairs of elements separated by a stride s are combined (e.g., via addition or maximization), where s starts at half the block size and is halved in each subsequent step. This process effectively constructs a binary tree-like reduction topology, progressively collapsing the data within the block. The final partial result is held by a designated thread (e.g., thread 0) and written back to global memory. After all partial results are generated by all thread blocks, another separate kernel is launched to reduce these intermediates into the final output.

2.2 Cascaded Reductions in Deep Learning

Cascaded reductions are pervasive in deep learning models, where their performance directly impacts end-to-end performance.

A canonical example is the safe softmax computation. It first computes the maximum value along a specified dimension (a max reduction), subtracts this maximum from the input, computes the sum of exponentials (a sum reduction), and finally normalizes the result. This process involves two consecutive reductions with strict data dependence, making it a prototypical instance of cascaded reductions. This pattern naturally extends to more complex model components:

- **Attention**. The entire forward pass of attention mechanism of Transformers [38] can be abstracted as GEMM + Softmax + GEMM, forming complex cascaded reductions that become a critical performance bottleneck.
- **MoE routing**. In Mixture-of-Experts (MoE) models, the router often employs a softmax + top-k pipeline: softmax normalizes the gating scores (involving max and sum reductions), followed by selecting the top-k experts.
- **FP8 Quant + GEMM**. In FP8 per-token quantization, dynamic scaling factors are computed for each token, often based on the absolute maximum value (an abs-max reduction). The quantized FP8 tensors are then used in GEMM operations, yielding a Quant + GEMM cascade.

2.3 Limitations of Previous Works

Existing deep learning compilers exhibit limitations in optimizing cascaded reductions. On the one hand, systems such as DNNFusion[24] and AStitch[49] lack dedicated fusion rules for these patterns. On the other hand, while frameworks like Bolt[42], Chimera[47], MCFuser[46], and SOUFFLE[41] may incidentally handle certain instances, such as the attention pattern, they do not formally model cascaded reductions as a structured computation pattern, nor do they perform in-depth analysis of their dependency and memory access characteristics. As a result, these compilers fail to generate loop-level fused kernels across multiple reduction stages, leading to suboptimal fusion strategies. In contrast, hand-optimized kernels such as FlashAttention[5] and FlashDecoding[6] achieve remarkable performance by leveraging tiling and online softmax techniques, but their implementations rely on expert-level GPU programming and manual scheduling. Moreover, their optimization logic is tightly coupled to specific computational patterns, making them difficult to generalize to other cascaded reduction structures, such as the softmax + top-k pipeline in MoE routing or the Quant + GEMM chain in FP8 computation. Together, these limitations highlight a compelling opportunity: the need for a compiler framework that not only formally characterizes the structure of cascaded reductions, but also enables automated, semantics-preserving fusion transformations, thereby bridging the gap between high performance and broad applicability.

3 Optimization Methods for Cascaded Reductions

This chapter presents a mathematical definition of the cascaded reductions and formally describes its typical implementation on AI accelerators—the chain of reduction trees. Subsequently, we derive a method that fuses chain of reduction trees into a single reduction tree. Building upon this, an incremental computation formulation is proposed.

3.1 Formal Definition of Cascaded Reductions

As shown in Figure 2, consider I cascaded reduction operations defined over a set $\mathbb{S}$, with input consisting of M data vectors $\{\boldsymbol{X}_1, \boldsymbol{X}_2, \ldots, \boldsymbol{X}_M\}$, each of length L_0, denoted as $\boldsymbol{X} \in \mathbb{S}^{M \times L_0}$. Let d_i be the output of the i-th reduction operation, which depends on the previous $i-1$ reduction results $\boldsymbol{D}_i = \{d_1, d_2, \ldots, d_{i-1}\}$. All reduction results form a

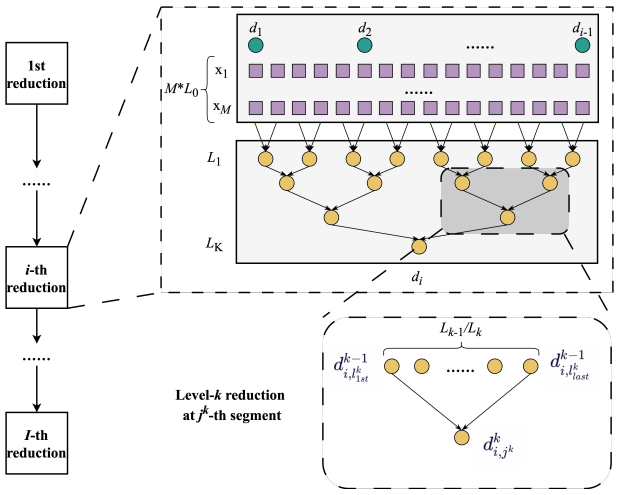


Figure 2. The formal definition of cascaded reductions

vector $\boldsymbol{D} = \{d_1, d_2, \ldots, d_N\}$ in dependency order. The i-th reduction expression can be formally defined as:

$$d_i = \bigoplus_{l=1}^{L_0} \mathcal{R}_i \, \mathrm{F}_i(\boldsymbol{X}[l], \boldsymbol{D}_i), \tag{1}$$

where:

- $\mathcal{R}_i$ denotes the i-th reduction operation, whose underlying associative operator is $\oplus_i$;
- $\boldsymbol{X}[l] = \{\boldsymbol{X}_1[l], \boldsymbol{X}_2[l], \ldots, \boldsymbol{X}_M[l]\}$ represents the set of input elements at position l;
- $\mathrm{F}_i : \mathbb{S}^M \times \mathbb{S}^{i-1} \to \mathbb{S}$ is the mapping function for the i-th reduction, operating on the input and dependent results.

3.1.1 Reduction Tree.

If the binary operator $\oplus_i$ satisfies the following properties:

- **Associativity**: $\forall s_1, s_2, s_3 \in \mathbb{S}, s_1 \oplus_i (s_2 \oplus_i s_3) = (s_1 \oplus_i s_2) \oplus_i s_3$;
- **Commutativity**: $\forall s_1, s_2 \in \mathbb{S}, s_1 \oplus_i s_2 = s_2 \oplus_i s_1$;

then the reduction operation can be split into multiple independent segments computed in parallel, whose outputs are later combined via the associative operator. This property allows the reduction process to be organized into a reduction tree structure, formally defined as follows.

Suppose the i-th reduction operation structured into K levels, where the output length at level k is denoted by L_k ($k = 1, 2, \ldots, K$), satisfying $L_0 > L_1 > \cdots > L_K = 1$.

For the j^1-th segment at the first level ($k = 1$), its output is defined as:

$$d^1_{i,j^1} = \bigoplus_{l=l^1_{1st}}^{l^1_{last}} \mathcal{R}_i \, \mathrm{F}_i(\boldsymbol{X}[l], \boldsymbol{D}_i), \tag{2}$$

where $l^1_{1st} = (j^1 - 1)\frac{L_0}{L_1} + 1$ and $l^1_{last} = j^1 \frac{L_0}{L_1}$ represent the index range of the input for this segment.

For level $k > 1$, each level k partitions the L_{k-1} outputs of the previous level into segments of length L_{k-1}/L_k, which serve as the inputs of the current level. Let d^k_{i,j^k} denote the output of the j^k-th segment at level k. Then:

$$d^k_{i,j^k} = \bigoplus_{j^{k-1}=l^k_{1st}}^{l^k_{last}} \mathcal{R}_i \; d^{k-1}_{i,j^{k-1}}, \quad k > 1, \tag{3}$$

where $l^k_{1st} = (j^k - 1)\frac{L_{k-1}}{L_k} + 1$ and $l^k_{last} = j^k \frac{L_{k-1}}{L_k}$.

3.1.2 Chain of Reduction Trees.

It is important to note that the i-th reduction depends on the results $\boldsymbol{D}_i$ of the preceding $i-1$ reductions, that is, it must wait for all roots of the preceding reduction trees before it can begin. This strong sequential dependency forces the reduction trees to execute in a chain-like order, forming a **Chain of Reduction Trees**.

3.2 Fused Cascaded Reductions

As shown in Figure 3, this subsection derives a fusion transformation to break inter-reduction dependencies and enable cross-reduction expression fusion.

3.2.1 Fusion Feasibility Conditions.

To support cross-reduction fusion, we require the i-th reduction operation (1) to satisfy the following three conditions:

- **Decomposability**: There exist functions $\mathrm{G}_i : \mathbb{S}^M \to \mathbb{S}$, $\mathrm{H}_i : \mathbb{S}^{i-1} \to \mathbb{S}$, and a binary operator $\otimes_i$ such that:
$$\mathrm{F}_i(\boldsymbol{X}[l], \boldsymbol{D}_i) = \mathrm{G}_i(\boldsymbol{X}[l]) \otimes_i \mathrm{H}_i(\boldsymbol{D}_i). \tag{4}$$
- **Algebraic Structure**: $(\mathbb{S}, \otimes_i)$ forms a commutative monoid, i.e., it satisfies:
  - Associativity: $\forall s_1, s_2, s_3 \in \mathbb{S}, s_1 \otimes_i (s_2 \otimes_i s_3) = (s_1 \otimes_i s_2) \otimes_i s_3$
  - Commutativity: $\forall s_1, s_2 \in \mathbb{S}, s_1 \otimes_i s_2 = s_2 \otimes_i s_1$
  - Existence of identity: $\exists e \in \mathbb{S}$ such that $\forall s \in \mathbb{S}, e \otimes_i s = s \otimes_i e = s$
- **Distributivity**: The reduction operator $\oplus_i$ distributes over $\otimes_i$:
$$(s_1 \oplus_i s_2) \otimes_i s_3 = (s_1 \otimes_i s_3) \oplus_i (s_2 \otimes_i s_3). \tag{5}$$

These conditions are widely satisfied in common reductions, such as GEMM and safe softmax.

3.2.2 Derivation of Fusion Expressions.

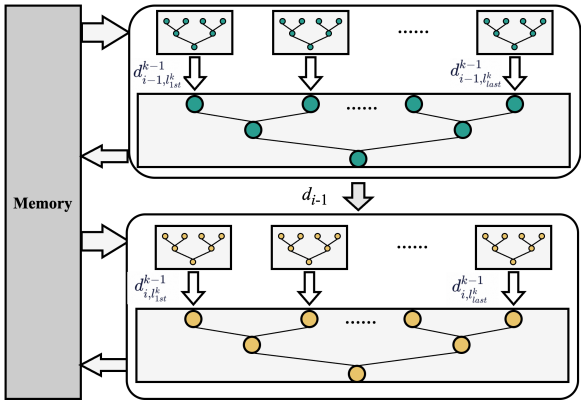
As illustrated in Figure 3b, let $\hat{d}^k_{i,j^k}$ denote the output of the j^k-th segment at level k of the i-th reduction. This output depends on the level-k outputs of the preceding $i-1$ reductions, denoted by $\hat{\boldsymbol{D}}^k = \{\hat{d}^k_{1,j^k}, \ldots, \hat{d}^k_{i-1,j^k}\}$.

At the first level, the output of the j^1-th segment is:

$$\hat{d}^1 = \bigoplus_{l=l^1_{1st}}^{l^1_{last}} \mathcal{R}_i \, \mathrm{G}_i(\boldsymbol{X}[l]) \otimes_i \mathrm{H}_i(\hat{\boldsymbol{D}}^1). \tag{6}$$

Using Equation (5), $\mathrm{H}_i(\hat{\boldsymbol{D}}^1)$ can be factored out:

$$\hat{d}^1 = \left(\bigoplus_{l=l^1_{1st}}^{l^1_{last}} \mathcal{R}_i \, \mathrm{G}_i(\boldsymbol{X}[l]) \right) \otimes_i \mathrm{H}_i(\hat{\boldsymbol{D}}^1). \tag{7}$$



(a) Cascaded reductions before fusion

(b) Cascaded reductions after fusion

Figure 3. Comparison of computation and memory access patterns in the cascaded reductions before and after fusion. (a) Unfused cascaded reductions: each reduction re-loads the input data and accesses the results from prior reductions. (b) Fused cascaded reductions: the input is loaded only once, and memory accesses to results of preceding reductions are eliminated.

If $\mathrm{H}_i(\hat{\boldsymbol{D}}^1)$ is invertible under $(\mathbb{S}, \otimes_i)$, with inverse denoted by $\mathrm{H}_i(\hat{\boldsymbol{D}}^1)^{-1}$, then:

$$\mathop{\mathcal{R}_i}_{l=l^1_{1st}}^{l^1_{last}} \mathrm{G}_i(\boldsymbol{X}[l]) = \hat{d}^1 \otimes_i \mathrm{H}_i(\hat{\boldsymbol{D}}^1)^{-1}. \tag{8}$$

Furthermore, the output of the j^2-th segment at level 2 is:

$$\hat{d}^2 = \mathop{\mathcal{R}_i}_{j^1=l^2_{1st}}^{l^2_{last}} \left(\mathop{\mathcal{R}_i}_{l=l^1_{1st}}^{l^1_{last}} \mathrm{G}_i(\boldsymbol{X}[l]) \otimes_i \mathrm{H}_i(\hat{\boldsymbol{D}}^2) \right). \tag{9}$$

Substituting Equation (8) into Equation (9):

$$\hat{d}^2 = \mathop{\mathcal{R}_i}_{j^1=l^2_{1st}}^{l^2_{last}} \left(\hat{d}^1 \otimes_i \mathrm{H}_i(\hat{\boldsymbol{D}}^1)^{-1} \otimes_i \mathrm{H}_i(\hat{\boldsymbol{D}}^2) \right). \tag{10}$$

Generalizing to level $k > 1$, the local result at segment j^k can be expressed as:

$$\hat{d}^k = \mathop{\mathcal{R}_i}_{j^{k-1}=l^k_{1st}}^{l^k_{last}} \left(\hat{d}^{k-1} \otimes_i \mathrm{H}_i(\hat{\boldsymbol{D}}^{k-1})^{-1} \otimes_i \mathrm{H}_i(\hat{\boldsymbol{D}}^k) \right). \tag{11}$$

Noted that $\mathrm{H}_i(\hat{\boldsymbol{D}}^{k-1})$ must be invertible under $(\mathbb{S}, \otimes_i)$. For the non-invertible case, we present a correction to $\mathrm{H}_i(\hat{\boldsymbol{D}}^{k-1})$ in the Appendix A.1, ensuring that the Equation 11 remains valid.

In the above expression, $\hat{d}^k$ depends on the outputs at the same level $\hat{\boldsymbol{D}}^k$ from the preceding $i-1$ reductions, rather than on the outputs $\boldsymbol{D}_i$ from the last level K. This implies:

- Expressions from multiple reduction trees at the same level k can be merged into a single reduction tree;
- Input data $\boldsymbol{X}$ and dependent results $\hat{\boldsymbol{D}}^k$ can be cached on-chip and reused, avoiding redundant memory accesses.

3.3 Incremental Computation Form

Although the fused expression (11) eliminates redundant memory accesses, it still requires the complete caching of the outputs $\hat{d}^{k-1}_{i,j^{k-1}}$ from the previous level $k-1$, as well as $\hat{\boldsymbol{D}}^{k-1}_{i,j^{k-1}}$ and $\hat{\boldsymbol{D}}^{k}_{i,j^{k-1}}$, before computation can proceed.

To address this, we derive an **incremental computation form**. As shown in Figure 4, the core idea of this method is to immediately update the current-level output upon completion of preceding level's computation.

3.3.1 Derivation of Incremental Computation.

Consider the reduction process for the j^k-th segment at level k, $k > 1$. Suppose the first $L-1$ inputs have been processed ($1 \le L \le L_{k-1}/L_k$), and the partial result is denoted by $\hat{d}^k[L-1]$, given as:

$$\hat{d}^k[L-1] = \mathop{\mathcal{R}_i}_{j^{k-1}=l^k_{1st}}^{l^k_{1st}+L-1} \left(\hat{d}^{k-1} \otimes_i \mathrm{H}_i(\hat{\boldsymbol{D}}^{k-1})^{-1} \otimes_i \mathrm{H}_i(\hat{\boldsymbol{D}}^k[L-1]) \right), \tag{12}$$

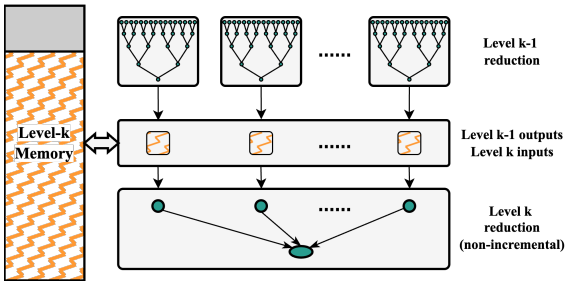
where $\hat{\boldsymbol{D}}^k[L-1]$ represents the set of partial results from the preceding $i-1$ reductions at the same level and segment, computed over $L-1$ input elements.

Same as Equation (8), we have:

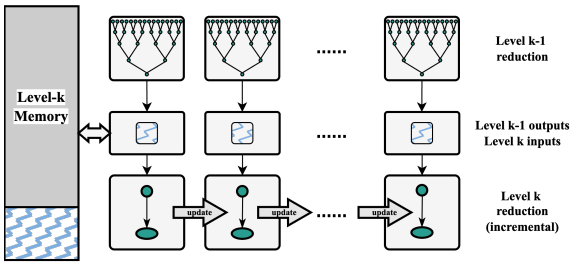
$$\mathop{\mathcal{R}_i}_{j^{k-1}=l^k_{1st}}^{l^k_{1st}+L-1} \hat{d}^{k-1} \otimes_i \mathrm{H}_i(\hat{\boldsymbol{D}}^{k-1})^{-1} = \hat{d}^k[L-1] \otimes_i \mathrm{H}_i(\hat{\boldsymbol{D}}^k[L-1])^{-1}. \tag{13}$$

When processing the L-th input, the partial result becomes:

$$\hat{d}^k[L] = \mathop{\mathcal{R}_i}_{j^{k-1}=l^k_{1st}}^{l^k_{1st}+L-1} \left(\hat{d}^{k-1} \otimes_i \mathrm{H}_i(\hat{\boldsymbol{D}}^{k-1})^{-1} \otimes_i \mathrm{H}_i(\hat{\boldsymbol{D}}^k[L]) \right). \tag{14}$$



(a) Non-incremental mode. Memory usage grows linearly with the input length.



(b) Incremental mode. A constant memory usage regardless of input length.

Figure 4. Comparison between non-incremental and incremental computation. (a) Non-incremental computation: the k-th level reduction must wait until all inputs are available before execution and on-chip memory consumption grows with the input length. (b) Incremental computation: the result at level k can be updated immediately upon arrival of new input, maintaining constant on-chip memory footprint.

Splitting $\hat{d}^k[L]$ into the first $L-1$ terms and the L-th term, and using Equation (13), we obtain:

$$\begin{aligned}\hat{d}^k[L] =& \hat{d}^k[L-1] \otimes_i \mathrm{H}_i(\hat{\boldsymbol{D}}^k[L-1])^{-1} \otimes_i \mathrm{H}_i(\hat{\boldsymbol{D}}^k[L]) \oplus_i \\ & \hat{d}^{k-1} \otimes_i \mathrm{H}_i(\hat{\boldsymbol{D}}^{k-1})^{-1} \otimes_i \mathrm{H}_i(\hat{\boldsymbol{D}}^k[L]).\end{aligned} \tag{15}$$

In particular, when $L = L_{k-1}/L_k$, we have $l^k_{1st}+L-1 = l^k_{last}$, and $\hat{d}^k[L]$ becomes the $\hat{d}^k$.

For the first-level reduction, where inputs are $\boldsymbol{X}[l]$, a similar derivation applies. Suppose the first $L-1$ elements have been processed; then the incremental update is:

$$\begin{aligned}\hat{d}^1[L] =& \hat{d}^1[L-1] \otimes_i \mathrm{H}_i(\hat{\boldsymbol{D}}^1[L-1])^{-1} \otimes_i \mathrm{H}_i(\hat{\boldsymbol{D}}^1[L]) \\ & \oplus_i \mathrm{G}_i(\boldsymbol{X}[L]) \otimes_i \mathrm{H}_i(\hat{\boldsymbol{D}}^1[L]).\end{aligned} \tag{16}$$

Under this incremental mechanism, the computation of $\hat{d}^k[L]$ depends only on the previous state $\hat{d}^k[L-1]$ and the current segment output $\hat{d}^{k-1}$ (or $\boldsymbol{X}[L]$). This storage overhead is constant-sized. Thus, the storage complexity is reduced from $O(L_{k-1})$ to $O(1)$.

3.4 Case Study: FP8 PerToken Quant + GEMM

We present a concrete application of the proposed methodology using FP8 per-token quantization followed by GEMM, a common cascaded reduction pattern employed to reduce the computational and memory overhead in serving large language models.

For a given token A to be quantized, the computation is expressed as:

$$\begin{aligned} m = d_1 &= \max_{l=1}^{L_0} |A[l]|, \\ c = d_2 &= \sum_{l=1}^{L_0} \frac{\mathrm{MAX} * A[l]}{m} * W[l], \end{aligned} \tag{17}$$

where $W[l]$ denotes the l-th row of the weight matrix W, and MAX is the maximum value in the FP8 format.

Under the fusion feasibility conditions (see Section 3.2.1), we define the functions $\mathrm{G}_i(\cdot)$, $\mathrm{H}_i(\cdot)$, and binary operators $\otimes_i$ for each reduction as follows:

$$\begin{aligned} &\mathrm{G}_1(A[l]) = |A[l]|, \quad \mathrm{H}_1(\cdot) = 0, \quad \otimes_1 = +, \\ &\mathrm{G}_2(A[l], W[l]) = A[l] * W[l], \quad \mathrm{H}_2(m) = \frac{\mathrm{MAX}}{m}, \quad \otimes_2 = *. \end{aligned} \tag{18}$$

3.4.1 Fused Expression.

Substituting Equation (18) into the general fusion expression (Equation (11)), we obtain the fused computation form for the level-k reduction ($k > 1$) at the j^k-th segment:

$$\begin{aligned} \hat{m}^k &= \max_{j^{k-1}=l^k_{1st}}^{l^k_{last}} \hat{m}^{k-1}, \\ \hat{c}^k &= \sum_{j^{k-1}=l^k_{1st}}^{l^k_{last}} \hat{c}^{k-1} * \frac{\hat{m}^{k-1}}{\hat{m}^k}. \end{aligned} \tag{19}$$

For $k = 1$, according to Equation (6), we have:

$$\begin{aligned} \hat{m}^1 &= \max_{l=l^1_{1st}}^{l^1_{last}} |A[l]|, \\ \hat{c}^1 &= \sum_{l=l^1_{1st}}^{l^1_{last}} \frac{\mathrm{MAX} * A[l]}{\hat{m}^1} * W[l]. \end{aligned} \tag{20}$$

3.4.2 Incremental Computation Form.

Substituting Equation (18) into the Equation (15), we obtain the incremental computation form for the level-k reduction ($k > 1$) on the j^k-th input segment:

$$\begin{aligned} \hat{m}^k[L] &= \max(\hat{m}^k[L-1], \hat{m}^{k-1}), \\ \hat{c}^k[L] &= \hat{c}^k[L-1] * \frac{\hat{m}^k[L-1]}{\hat{m}^k[L]} + \hat{c}^{k-1} * \frac{\hat{m}^{k-1}}{\hat{m}^k[L]}. \end{aligned} \tag{21}$$

Based on Equation (16), the incremental form at the first level can be derived as:

$$\hat{m}^1[L] = \max(\hat{m}^1[L-1], |a[L]|),$$
$$\hat{c}^1[L] = \hat{c}^1[L-1] * \frac{\hat{m}^1[L-1]}{\hat{m}^1[L]} + \frac{\mathrm{MAX} * a[L]}{\hat{m}^1[L]} * w[L]. \tag{22}$$

More cases, including FlashAttention, are provided in the Appendix A.2 for reference.

4 RedFuser

Building upon the theoretical methodology, this chapter presents RedFuser, an optimization framework designed to fuse cascaded reductions. To minimize development effort and avoid redundant infrastructure work, we build RedFuser on top of the TVM[3] compiler stack, leveraging its mature frontend and runtime ecosystem. Specifically, we use TVM's frontend to lower models from mainstream deep learning frameworks (e.g., PyTorch[1]) into a unified Relax[17] computational graph, where we identify and partition the graph into cascaded reduction subgraphs and other non-reduction components. The cascaded reduction subgraphs are then processed by RedFuser. We first traverse their abstract syntax trees (ASTs) to convert each reduction operation into a formal mathematical expression. Next, using our automatic fusion algorithm, we extract the key functional components $\mathrm{G}_i(\cdot)$ and $\mathrm{H}_i(\cdot)$. Finally, guided by our fused reduction programming model, we generate high-performance, memory-efficient kernels tailored for modern GPU-based AI accelerators.

4.1 Mathematical Representation of Cascaded Reductions

Given a Relax[17] subgraph containing a cascaded reduction pattern, we lower it into TensorIR (TIR) using TVM's standard lowering pipeline. At the TIR level, we first apply a series of preprocessing transformations, including function inlining and loop reordering to normalize and optimize the computational structure. We then traverse the TIR's abstract syntax tree (AST) via a dedicated visitor to extract a formal mathematical expression that precisely characterizes the reduction chain. This expression captures both the computational semantics and data dependencies across consecutive reductions, serving as the input to our automatic fusion algorithm.

4.2 Automatic Fusion Algorithm

Building upon theoretical methodology in Chapter. 3, this subsection proposes an engineering-practical **Automatic Cascaded Reductions Fusion (ACRF)** algorithm that automatically identifies fusion-capable expressions and generates fused expression.

4.2.1 Domain-Specific Decomposition Feasibility.

The key to fusion lies in decomposing the reduction function $\mathrm{F}_i(\mathbf{x}[l], \mathbf{d}_i)$ into the form that satisfies the conditions specified in Chapter 3.2.1. In general function spaces, this decomposition has an enormous search space and no closed-form solution. However, for machine learning workloads, we can introduce the following domain-specific assumptions to significantly narrow the search space:

- The set $\mathbb{S}$ is the set of real numbers $\mathbb{R}$;
- The underlying operators $\oplus_i$ for common reduction operations $\mathcal{R}_i$ are limited and primarily include:
  - Summation ($\sum$, $\oplus_i = +$)
  - Product ($\prod$, $\oplus_i = *$)
  - Extrema (max, min)

Table 1 lists common reduction operations in machine learning along with their corresponding $\oplus_i$ and compatible $\otimes_i$.

Table 1. Common reduction operations in machine learning and their corresponding binary operators

Reduction Operation $\mathcal{R}_i$	$\oplus_i$	$\otimes_i$
Max, ArgMax, TopK, etc.	max	+
Min, ArgMin, etc.	min	+
Sum, Inner Product, Matrix Multiply, etc.	+	*
Prod[1]	+	*

Given a reduction operation $\mathcal{R}_i$, its corresponding $\otimes_i$ can be directly determined from the Table 1. This simplifies the decomposition problem to: do there exist functions $\mathrm{G}_i(\cdot)$ and $\mathrm{H}_i(\cdot)$ such that $\mathrm{F}_i(\mathbf{x}[l], \mathbf{d}_i) = \mathrm{G}_i(\mathbf{x}[l]) \otimes_i \mathrm{H}_i(\mathbf{d}_i)$?

4.2.2 Automatic Function Decomposability.

To determine the feasibility of such a decomposition, we introduce the **Fixed-Point Analysis Method**. Select a constant input point $(\mathbf{x}[l]_0, \mathbf{d}_{i0})$, requiring that $\mathrm{F}_i(\mathbf{x}[l]_0, \mathbf{d}_{i0})$ has an inverse under $(\mathbb{R}, \otimes_i)$ (e.g., non-zero when $\otimes_i = *$).

If the following identity holds:

$$\mathrm{F}_i(\mathbf{x}[l], \mathbf{d}_i) \otimes_i \mathrm{F}_i(\mathbf{x}[l]_0, \mathbf{d}_{i0}) = \mathrm{F}_i(\mathbf{x}[l], \mathbf{d}_{i0}) \otimes_i \mathrm{F}_i(\mathbf{x}[l]_0, \mathbf{d}_i) \tag{23}$$

then F_i is decomposable, with:

$$\mathrm{G}_i(\mathbf{x}[l]) = \mathrm{F}_i(\mathbf{x}[l], \mathbf{d}_{i0}) \tag{24}$$
$$\mathrm{H}_i(\mathbf{d}_i) = \mathrm{F}_i(\mathbf{x}[l]_0, \mathbf{d}_i) \otimes_i \mathrm{F}_i(\mathbf{x}[l]_0, \mathbf{d}_{i0})^{-1} \tag{25}$$

Proof: Rearranging Equation (23) yields:

$$\mathrm{F}_i(\mathbf{x}[l], \mathbf{d}_i) = \mathrm{F}_i(\mathbf{x}[l], \mathbf{d}_{i0}) \otimes_i \big(\mathrm{F}_i(\mathbf{x}[l]_0, \mathbf{d}_i) \otimes_i \mathrm{F}_i(\mathbf{x}[l]_0, \mathbf{d}_{i0})^{-1}\big) \tag{26}$$

The right-hand side consists of two terms: the first depends only on $\mathbf{x}[l]$ (defining G_i), and the second only on $\mathbf{d}_i$ (defining H_i). Hence, the decomposition is valid.

[1] $\prod \mathrm{F}_i$ can be transformed into summation via $\prod \mathrm{F}_i = \mathrm{sgn}(\cdot) 2^{\sum \log_2 |\mathrm{F}_i|}$.

4.2.3 Automatic Fusion Algorithm Workflow.

As shown in Algorithm 1, the ACRF algorithm proceeds as follows:

Algorithm 1 Automatic Cascaded Reductions Fusion (ACRF)

Require: Function $\mathrm{F}_i(\mathbf{x}[l], \mathbf{d}_i)$, reduction operator $\mathcal{R}_i$
Ensure: Fused and incremental expressions, or failure flag

```
Determine ⊗_i by looking up Table 1 based on ⊕_i
Select a fixed point (x[l]_0, d_i0) such that F_i(x[l]_0, d_i0) ≠ 0 (if ⊗_i = ∗)
if Equation (23) does not hold then
    return NotFusable
end if
Compute G_i(x[l]) ← F_i(x[l], d_i0)
Compute H_i(d_i) ← F_i(x[l]_0, d_i) ⊗_i F_i(x[l]_0, d_i0)^{-1}
Instantiate Equation (6), (11), (16) and (15) using computed G_i(x[l]) and H_i(d_i)
return Instantiated Equation (6), (11), (16) and (15)
```

The algorithm can be implemented using symbolic computation tools (e.g., SymPy[22]).

4.3 Programming Model on GPU

The classic programming model for reduction on GPUs follows a four-level hierarchy: **intra-thread**, **intra-warp**, **intra-block**, and **inter-block**. In the context of hierarchical reduction tree, each level's output length corresponds to a specific granularity of execution resources: L_1 is the number of threads, L_2 the number of warps, L_3 the number of CTAs (Cooperative Thread Arrays), and $L_4 = 1$. We adopt this established programming model to implement fused reduction operations on GPU.

Notably, the non-incremental mode, constrained by on-chip resources, struggles to support reduction of long sequences within a single CTA; whereas the incremental mode, through streaming updates, allows the system to partition inputs into segments of arbitrary length within limited storage, thereby enabling control over parallelism—another critical factor affecting operator performance. Leveraging the key advantage of incremental computation, we propose two implementation strategies that adapt to varying hardware resource constraints and parallelization requirements.

- **Single-Segment Strategy**: The Single-Segment strategy adopts the incremental computation mode, so the entire reduction can be completed within a single CTA even when the input length exceeds the capacity of a single CTA. In this way, it no longer suffers from hardware resource limitations and avoids the extra overhead of inter-block communication and synchronization.
- **Multi-Segment Strategy**: The Multi-Segment strategy builds on the Single-Segment strategy by partitioning the input into multiple segments that are processed in parallel by different CTAs, and then using the Equation (11) to merge their partial results into the final output. This strategy improves hardware utilization under varying workloads, particularly in low-concurrency scenarios.

While both strategies offer distinct advantages, their performance is heavily dependent on workload characteristics and hardware constraints. To achieve optimal performance across diverse scenarios, RedFuser simultaneously generates TIR representations for both strategies from the fused computation expressions, enabling subsequent code generation.

4.4 Code Generation

After converting fused expressions into TIR, RedFuser performs **Tensorization**, a transformation from scalar-level to tile-level IR that bridges high-level computation with efficient hardware execution. This process consists of three stages:

- **Blockization**: Restructures scalar loop nests into independently schedulable block-level tiles.
- **Block-level Buffer Management**: For each tile, allocates and optimizes on-chip buffers by (i) inserting explicit Load/Store statements for global I/O, (ii) inferring buffer scopes (e.g., register or shared memory), and (iii) compacting buffer size to the tile's minimal footprint.
- **Conversion to TileOp**: Maps tiles to standardized TileOps, enabling TileLang [40] code generation (see Appendix A.3).

Following Tensorization, RedFuser applies **Parallelization** to the tile-level IR by binding each block-level tile to a distinct block index (e.g., `blockIdx.x`), producing a TileLang program. RedFuser leverages TileLang to apply three key optimizations:

- **Thread-level Mapping**: Partitions block tiles across threads with explicit assignment of per-thread work and hardware resources.
- **Software Optimization**: Employs various techniques to hide latency. Key methods include analyzing TileOp dependencies to construct software pipelines, and splitting memory and compute tasks across warps.
- **Hardware-aware Implementations**: Selects architecture-optimized code paths. For example, `copy` uses `cp.async` (Ampere) or TMA (Hopper) with automatic vectorization and bank conflict avoidance, while `gemm` maps to MMA (Ampere) or WGMMA (Hopper) for peak throughput.

With TileLang, RedFuser automatically generates efficient GPU kernels. To ensure portable performance across diverse platforms and workload characteristics, the framework incorporates **Auto-Tuning** mechanism that optimizes key parameters, including block tile size, threads per block, software pipeline depth, and (in the Multi-Segment strategy) the number of segments. We construct an empirical search space for these parameters and perform runtime configuration selection, ensuring that the generated kernels are highly optimized.

Implementation details are provided in Appendix A.4, using **FlashAttention** and **FlashDecoding** as illustrative examples.

5 Performance Evaluation

This chapter evaluates the performance of RedFuser and compares it against state-of-the-art deep learning compilers.

5.1 Experimental Setup

Platform. We evaluate RedFuser on two modern GPU architectures: a 24GB NVIDIA A10 GPU and an 80GB NVIDIA H800 GPU. All experiments are conducted under Ubuntu 22.04 with CUDA 12.8.

Workloads. We evaluate RedFuser on four representative subgraphs. The selected workloads are:

- **Attention** module. We consider two variants, Multi-Head Attention (**MHA**) from BERT [7], ViT [8] and LLaMA-65B [37], and Multi-Latent Attention (**MLA**) from DeepSeek-R1 [12].
- **MoE routing** module. The routing function consists of a GEMM for calculating scores for each expert, followed by a softmax + topk to select experts. We use different parameter configurations in different models (Switch Transformer [10], ERNIE [2], DeepSeek-V2-Lite [21] and Qwen3 [44]) for the experiment.
- **FP8 PerToken Quant + GEMM** module. This module first computes dynamic scaling factors via reduction operations, then applies quantization and performs FP8 GEMM calculation. We select configurations from existing FP8 quantized MoE models (ERNIE [2], DeepSeek-R1 [12] and Qwen3 [44]) for testing.

The detailed configurations of all modules are shown in Appendix A.5.

We also conduct experiments on non-ML workloads, such as variance computation. Details on the experimental setup and results are provided in Appendix A.6.

Baselines. We compare RedFuser against the following strong baselines.

- **PyTorch (v2.7) Eager**. The native PyTorch implementation serves as the primary baseline.
- **PyTorch Dynamo**. We use `torch.compile` with the default backend (Inductor) to enable graph capture, operator fusion and code generation. The Inductor backend will generate Triton [36] code on GPUs.
- **TVM (v0.21)**. We compile all workloads using TVM's Relax [17] frontend and perform the default optimization pipeline. In particular, we do not enable TVM's CUTLASS [35] or FlashInfer [45] backends in order to compare TVM's own operator fusion capabilities.
- **Hand-Optimized Libraries**. We evaluate FlashAttention2 [4] and FlashMLA [15], which provide highly optimized CUDA kernels for MHA and MLA patterns.

5.2 Subgraph Performance

5.2.1 MHA and MLA.

Figure 5a and 5b show the normalized performance of MHA and MLA workloads, with all measurements reported relative to PyTorch Eager.

For MHA, the fused kernel generated by RedFuser achieves performance on par with hand-optimized implementations, averaging 1.09× that of FlashAttention2, while outperforming it in several configurations (H1-H5). On the LLaMA-65B configuration, RedFuser delivers 2.8× and 2.6× speedup over PyTorch Dynamo and TVM. For MLA, our generated kernel achieves 102% of the performance of FlashMLA, while significantly outperforming PyTorch Dynamo and TVM by 2.4× and 8.7×. The performance gains stem from our fine-grained fusion of the full attention reduction chain.

5.2.2 MoE routing and Quant GEMM.

Due to lack of widely adopted hand-optimized implementations, we compare RedFuser only against general-purpose compilers. Figure 5c and 5d present the performance of MoE routing and Quant+GEMM, normalized to PyTorch Eager.

RedFuser achieves significant speedups across both workloads. In MoE routing, it delivers 1.7× and 6.6× speedup over PyTorch Dynamo and TVM, respectively. For Quant + GEMM, the improvements are even more pronounced, reaching 3.4× over PyTorch Dynamo and 12.1× over TVM.

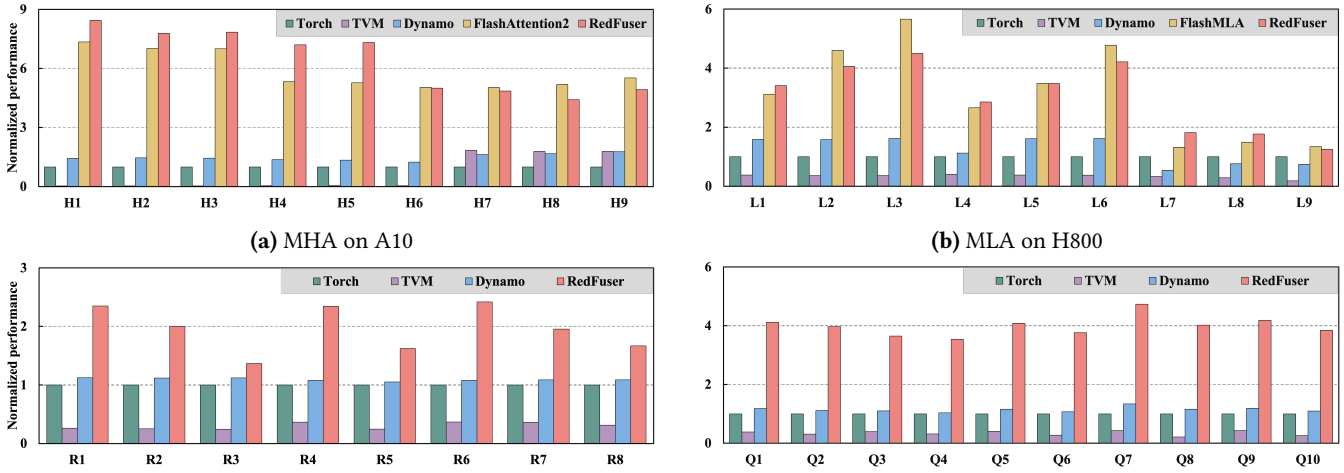
These experiments confirm that RedFuser exhibits strong performance across various representative workloads, rivaling hand-optimized implementations. We evaluate RedFuser on multiple hardware platforms to assess its portability. Detailed results are provided in Appendix A.7.

5.3 Latency of Different-Level Fusion

We evaluate four fusion strategies for the safe softmax operator – *intra-thread*, *intra-warp*, *intra-block*, and *inter-block* – over input sizes from 1K to 8K to assess the impact of fusion at different levels on the performance of the fused operator. All latency measurements are normalized with respect to the unfused kernels. Figure 6a reports the normalized performance of different fusion strategies.

The results show that all fusion strategies reduce latency compared to the unfused baseline. And the latency ordering is: intra-block < inter-block ≈ intra-warp < intra-thread. The performance differences are analyzed as follows:

As shown in Figure 7, fusing cascaded reductions eliminates redundant memory accesses, yielding performance gains across all levels. However, as Equation (11) shows, fusion at level k requires correcting intermediate outputs $\hat{d}^k$ of length L_k, introducing computation overhead linear in L_k. Fortunately, in the fused reduction tree at level k ($k < K$), subtrees ② and ③ are data-independent, enabling instruction-level parallelism and overlap between memory access and computation. As k increases, the deeper subtree ③ provides longer computation paths, enhancing latency hiding.



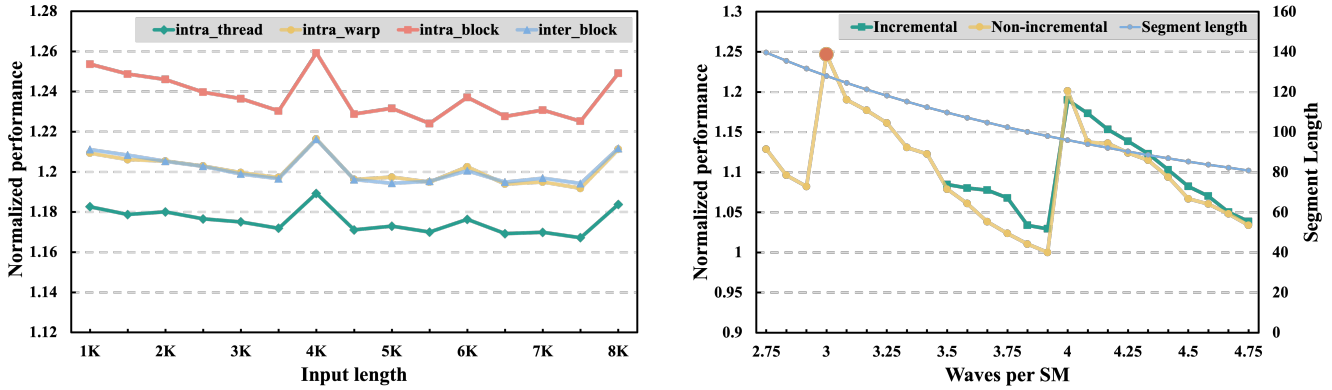
(a) MHA on A10

(b) MLA on H800

(c) MoE routing on A10

(d) FP8 PerToken Quant + GEMM on H800

Figure 5. The normalized performance of fusing four selected modules on GPUs.



(a) Normalized performance comparison of kernels fused at different levels. Intra-block fusion achieves the best performance.

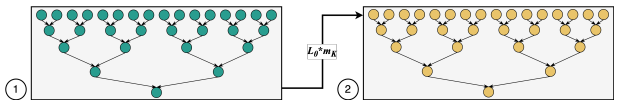
(b) Normalized performance of incremental and non-incremental computation across different parallelism levels.

Figure 6. Performance comparison results. (a) fusion at different levels. (b) incremental and non-incremental computation.

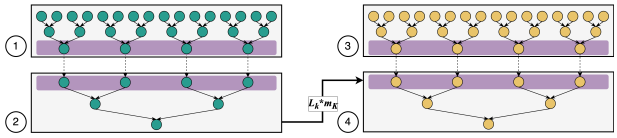
- **Intra-thread Fusion:** Requires L_1 corrections – the highest overhead – and the sub-reduction tree ③ spans only *level 1*, offering insufficient computation to hide memory latency. Thus, it delivers the smallest performance gain.
- **Intra-warp Fusion:** Spans *level-1* and *level-2*, increasing computational intensity and better hiding global memory latency. With lower correction cost than *intra-thread*, it achieves better performance.
- **Intra-block Fusion:** Introduces L_3 additional corrections but enables the best overlap between accessing d^K and computing subtree ③. Experimental results show that the latency-hiding benefit outweighs the overhead, resulting in the lowest overall latency.
- **Inter-block Fusion:** No correction is needed since $\hat{d}^K = d^K$. However, strict data dependencies prevent memory-computation overlap, limiting performance despite zero correction cost.

5.4 Latency of Incremental vs. Non-Incremental Mode

This section evaluates the performance of incremental computation versus non-incremental computation under varying degrees of parallelism. We use the attention computation pattern from the BERT-base model as our representative workload. With a fixed problem size, we systematically control the level of GPU parallelism by adjusting the sequence length of kv processed per CTA, thereby modulating the number of CTAs.



(a) Unfused cascaded reduction trees



(b) Fusion at level-k

Figure 7. Illustration of the impact of fusion at different levels on memory access patterns. (a) No fusion: d^K needs to be loaded L_0 times to compute $\mathrm{F}_i(\cdot)$. (b) Fusion at level k: reduces accesses to the final result d^K to L_k times.

In our experiments, we normalize all latency measurements with respect to the maximum observed execution latency across all configurations, which serves as the performance baseline. The normalized performance results are shown in Figure 6b. Based on the experimental findings, we draw the following conclusions:

- **On-chip cache capacity limits the applicability of non-incremental computation**: Non-incremental computation requires caching the complete previous reduction results in on-chip shared memory or registers, making its feasibility constrained by the SM's on-chip storage resources. In our experiments, the non-incremental approach only succeeds when the input sequence length is $\leq$ 112, corresponding to `Waves per SM` $\geq$ 3.5. In contrast, incremental computation avoids the need to retain full results on-chip, thereby eliminating strict constraints on segment length and enabling more flexible parallelism configurations.
- **Non-incremental computation exhibits superior performance at matched parallelism levels**: As shown in Equation (15), incremental computation introduces additional computational overhead due to the need to correct the result in each iteration. Consequently, under equivalent parallelism and hardware resource allocations, non-incremental computation outperforms its incremental counterpart, achieving lower latency and higher overall performance.
- **Parallelism level has a nonlinear impact on performance**: Operator performance exhibits local optima when `Waves per SM` takes on integer values, indicating peak SM resource utilization at these configurations. Notably, performance peaks at `Waves per SM` = 3, achieving up to a 1.25× speedup over the baseline configuration. However, this optimal configuration corresponds to a segment length that exceeds the on-chip memory capacity required by the non-incremental approach, and thus can only be realized using incremental computation. This highlights the necessity of incremental computation in specific high-performance configurations.

In summary, although incremental computation incurs additional computational overhead due to correction operations, it significantly reduces on-chip resource requirements and greatly expands the feasible range of parallelism configurations. This characteristic enlarges the optimization search space, enabling high-performance configurations even in low-parallelism and long-sequence scenarios—such as the decode phase of LLM inference, and thereby providing greater flexibility and potential for performance tuning in practical deployments.

6 Related Work

Parallel Reduction. The design of efficient parallel reduction algorithms on GPUs has been extensively studied. Harris[13] proposed a series of optimized algorithms for parallel reduction as early as 2007, based on a segmented reduction strategy that leverages shared memory and sequential addressing to maximize memory bandwidth utilization. Recent work[23] has explored the use of GPU Tensor Cores for accelerating parallel reduction. RedFuser builds upon well-established parallel reduction algorithms and extends them to cascaded reductions, under specific structural constraints and execution conditions.

Operator Fusion in DL Compilers. Many popular deep learning compilers have invested significant effort in supporting operator fusion, and some have already explored fusing reduction operators with other computational operators. Modern compilers such as TVM[3], XLA[32], and PyTorch Dynamo[1] support fusion of elementwise operators with a single reduction, based on either schedule templates (TVM), rule-based heuristics (XLA) or pattern matching (Dynamo). DNNFusion[24] and AStitch[49] classify operators into different types and apply fusion algorithms among them. Bolt[42] enables operator fusion by leveraging CUTLASS[35] templates to fuse sequences of GEMM and convolution operators. Chimera[47] and MCFuser[46] both focus on fusing memory-bound compute-intensive operators, such as chains of batched GEMM and convolution. SOUFFLE[41] adopts a top-down approach, aggressively fuses reduction operators with adjacent computation intensive operators. RedFuser formally defines the structural properties of cascade reductions, and presents a novel algorithm that enables cross-reduction expression fusion.

Hand-optimized Fusion. FlashAttention[5] is a hand-optimized implementation of the attention operator. Its key insight is to apply tiling to decompose the QK^T and PV GEMMs into smaller, manageable blocks, enabling intermediate values, such as attention scores and normalization factors to reside entirely in on-chip memory (e.g., shared memory) and thereby avoiding expensive round trips to global memory.

To ensure numerical correctness across tiles, FlashAttention employs an online softmax algorithm: it dynamically maintains running values of the maximum and sum of exponentials over processed tiles, and upon processing each new tile, rescales the accumulated results to incorporate the updated maximum before merging. This incremental update strategy allows FlashAttention to compute the exact softmax normalization in a memory-efficient manner. FlashDecoding[6] extends the tiling paradigm of FlashAttention to optimize long-context inference. Specifically, it partitions the KV cache into multiple chunks along the sequence axis, and applies FlashAttention independently within each chunk to compute local attention outputs and partial normalization statistics. These intermediate results are preserved and later aggregated by an additional reduction kernel, which performs a global max and sum reduction across all chunks to produce the final, numerically correct output. RedFuser derives an incremental computation form based on the fused reduction expressions. Under the guidance of our programming model, FlashAttention and FlashDecoding emerge as special cases of our framework. RedFuser not only subsumes these hand-optimized designs, but also unifies them within a single, automated compilation framework, enabling a transition from ad-hoc optimization to general-purpose fusion.

7 Conclusion

In this paper, we present a general fusion methodology for cascaded reduction operations, ubiquitous in AI models, which derives fused expressions and their corresponding incremental computation forms directly from the mathematical representations of reductions. Our approach eliminates redundant memory accesses and enables reductions of arbitrary length at any level, overcoming cache capacity limitations. Based on this methodology, we design an operator fusion framework named RedFuser that automatically analyzes cascaded reductions and generates high-performance, GPU-optimized fused kernels. Extensive experiments demonstrate that our generated kernels outperform state-of-the-art AI compilers and achieve performance on par with hand-written operators. While our methodology begins to bridge the gap between hand-crafted, highly specialized optimizations and general-purpose compiler automation, several challenges and opportunities remain. First, while fusion can reduce memory traffic, it introduces additional computational overhead and increases pressure on hardware resources such as register usage, making it suboptimal for certain workloads. We plan to develop a systematic cost model to guide fusion decisions. Second, our current approach applies only to operations satisfying specific conditions; extending it to more relaxed constraints is an important direction for future work. Finally, RedFuser has been primarily implemented and evaluated on GPU architectures. We aim to generalize the framework to support a broader range of AI accelerators, further enhancing its applicability in diverse computing environments.

References

[1] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, et al. 2024. Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 929–947.

[2] Baidu-ERNIE-Team. 2025. ERNIE 4.5 Technical Report. https://ernie.baidu.com/blog/publication/ERNIE_Technical_Report.pdf.

[3] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, et al. 2018. TVM: An automated End-to-End optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 578–594.

[4] Tri Dao. 2023. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691* (2023).

[5] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems* 35 (2022), 16344–16359.

[6] Tri Dao, Daniel Haziza, Francisco Massa, and Grigory Sizov. 2023. Flash-Decoding for long-context inference. https://pytorch.org/blog/flash-decoding/

[7] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. 4171–4186.

[8] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. 2020. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010. 11929* (2020).

[9] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, Dustin Podell, Tim Dockhorn, Zion English, Kyle Lacey, Alex Goodwin, Yannik Marek, and Robin Rombach. 2024. Scaling Rectified Flow Transformers for High-Resolution Image Synthesis. arXiv:2403.03206 [cs.CV] https://arxiv.org/abs/2403.03206

[10] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39.

[11] Mingyu Gao, Xuan Yang, Jing Pu, Mark Horowitz, and Christos Kozyrakis. 2019. TANGRAM: Optimized Coarse-Grained Dataflow for Scalable NN Accelerators. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems* (Providence, RI, USA) *(ASPLOS '19)*. Association for Computing Machinery, New York, NY, USA, 807–820. doi:10.1145/3297858.3304014

[12] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).

[13] Mark Harris et al. 2007. Optimizing parallel reduction in CUDA. *Nvidia developer technology* 2, 4 (2007), 70.

[14] Jiachen Jiang and Soroush Vosoughi. 2020. Not Judging a User by Their Cover: Understanding Harm in Multi-Modal Processing within Social Media Research. In *Proceedings of the 2nd International Workshop on Fairness, Accountability, Transparency and Ethics in Multimedia* (Seattle, WA, USA) *(FATE/MM '20)*. Association for Computing Machinery, New York, NY, USA, 6–12. doi:10.1145/3422841.3423534

[15] Shengyu Liu Jiashi Li. 2025. FlashMLA: Efficient MLA decoding kernels. https://github.com/deepseek-ai/FlashMLA.

[16] Sheng-Chun Kao, Suvinay Subramanian, Gaurav Agrawal, Amir Yazdanbakhsh, and Tushar Krishna. 2022. FLAT: An Optimized Dataflow for Mitigating Attention Bottlenecks. arXiv:2107.06419 [cs.LG] https://arxiv.org/abs/2107.06419

[17] Ruihang Lai, Junru Shao, Siyuan Feng, Steven Lyubomirsky, Bohan Hou, Wuwei Lin, Zihao Ye, Hongyi Jin, Yuchen Jin, Jiawei Liu, et al. 2025. Relax: composable abstractions for end-to-end dynamic machine learning. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 998–1013.

[18] Guillaume Lample and Alexis Conneau. 2019. Cross-lingual Language Model Pretraining. arXiv:1901.07291 [cs.CL] https://arxiv.org/abs/1901.07291

[19] Hang Le, Loïc Vial, Jibril Frej, Vincent Segonne, Maximin Coavoux, Benjamin Lecouteux, Alexandre Allauzen, Benoît Crabbé, Laurent Besacier, and Didier Schwab. 2020. FlauBERT: Unsupervised Language Model Pre-training for French. arXiv:1912.05372 [cs.CL] https://arxiv.org/abs/1912.05372

[20] Juhyoung Lee, Jinsu Lee, and Hoi-Jun Yoo. 2020. SRNPU: An Energy-Efficient CNN-Based Super-Resolution Processor With Tile-Based Selective Super-Resolution in Mobile Devices. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems* 10, 3 (2020), 320–334. doi:10.1109/JETCAS.2020.3014454

[21] Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Dengr, Chong Ruan, Damai Dai, Daya Guo, et al. 2024. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434* (2024).

[22] Aaron Meurer, Christopher P. Smith, Mateusz Paprocki, Ondřej Čertík, Sergey B. Kirpichev, Matthew Rocklin, AMiT Kumar, Sergiu Ivanov, Jason K. Moore, Sartaj Singh, Thilina Rathnayake, Sean Vig, Brian E. Granger, Richard P. Muller, Francesco Bonazzi, Harsh Gupta, Shivam Vats, Fredrik Johansson, Fabian Pedregosa, Matthew J. Curry, Andy R. Terrel, Štěpán Roučka, Ashutosh Saboo, Isuru Fernando, Sumith Kulal, Robert Cimrman, and Anthony Scopatz. 2017. SymPy: symbolic computing in Python. *PeerJ Computer Science* 3 (Jan. 2017), e103. doi:10.7717/peerj-cs.103

[23] Cristóbal A Navarro, Roberto Carrasco, Ricardo J Barrientos, Javier A Riquelme, and Raimundo Vega. 2020. GPU tensor cores for fast arithmetic reductions. *IEEE Transactions on Parallel and Distributed Systems* 32, 1 (2020), 72–84.

[24] Wei Niu, Jiexiong Guan, Yanzhi Wang, Gagan Agrawal, and Bin Ren. 2021. Dnnfusion: accelerating deep neural networks execution with advanced operator fusion. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. 883–898.

[25] NVIDIA. 2025. CUDA C++ Programming Guide. https://docs.nvidia.com/cuda/cuda-c-programming-guide

[26] OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, Irwan Bello, Jake Berdine, Gabriel Bernadett-Shapiro, Christopher Berner, Lenny Bogdonoff, Oleg Boiko, Madelaine Boyd, Anna-Luisa Brakman, Greg Brockman, Tim Brooks, Miles Brundage, Kevin Button, Trevor Cai, Rosie Campbell, Andrew Cann, Brittany Carey, Chelsea Carlson, Rory Carmichael, Brooke Chan, Che Chang, Fotis Chantzis, Derek Chen, Sully Chen, Ruby Chen, Jason Chen, Mark Chen, Ben Chess, Chester Cho, Casey Chu, Hyung Won Chung, Dave Cummings, Jeremiah Currier, Yunxing Dai, Cory Decareaux, Thomas Degry, Noah Deutsch, Damien Deville, Arka Dhar, David Dohan, Steve Dowling, Sheila Dunning, Adrien Ecoffet, Atty Eleti, Tyna Eloundou, David Farhi, Liam Fedus, Niko Felix, Simón Posada Fishman, Juston Forte, Isabella Fulford, Leo Gao, Elie Georges, Christian Gibson, Vik Goel, Tarun Gogineni,

Gabriel Goh, Rapha Gontijo-Lopes, Jonathan Gordon, Morgan Grafstein, Scott Gray, Ryan Greene, Joshua Gross, Shixiang Shane Gu, Yufei Guo, Chris Hallacy, Jesse Han, Jeff Harris, Yuchen He, Mike Heaton, Johannes Heidecke, Chris Hesse, Alan Hickey, Wade Hickey, Peter Hoeschele, Brandon Houghton, Kenny Hsu, Shengli Hu, Xin Hu, Joost Huizinga, Shantanu Jain, Shawn Jain, Joanne Jang, Angela Jiang, Roger Jiang, Haozhun Jin, Denny Jin, Shino Jomoto, Billie Jonn, Heewoo Jun, Tomer Kaftan, Łukasz Kaiser, Ali Kamali, Ingmar Kanitscheider, Nitish Shirish Keskar, Tabarak Khan, Logan Kilpatrick, Jong Wook Kim, Christina Kim, Yongjik Kim, Jan Hendrik Kirchner, Jamie Kiros, Matt Knight, Daniel Kokotajlo, Łukasz Kondraciuk, Andrew Kondrich, Aris Konstantinidis, Kyle Kosic, Gretchen Krueger, Vishal Kuo, Michael Lampe, Ikai Lan, Teddy Lee, Jan Leike, Jade Leung, Daniel Levy, Chak Ming Li, Rachel Lim, Molly Lin, Stephanie Lin, Mateusz Litwin, Theresa Lopez, Ryan Lowe, Patricia Lue, Anna Makanju, Kim Malfacini, Sam Manning, Todor Markov, Yaniv Markovski, Bianca Martin, Katie Mayer, Andrew Mayne, Bob McGrew, Scott Mayer McKinney, Christine McLeavey, Paul McMillan, Jake McNeil, David Medina, Aalok Mehta, Jacob Menick, Luke Metz, Andrey Mishchenko, Pamela Mishkin, Vinnie Monaco, Evan Morikawa, Daniel Mossing, Tong Mu, Mira Murati, Oleg Murk, David Mély, Ashvin Nair, Reiichiro Nakano, Rajeev Nayak, Arvind Neelakantan, Richard Ngo, Hyeonwoo Noh, Long Ouyang, Cullen O'Keefe, Jakub Pachocki, Alex Paino, Joe Palermo, Ashley Pantuliano, Giambattista Parascandolo, Joel Parish, Emy Parparita, Alex Passos, Mikhail Pavlov, Andrew Peng, Adam Perelman, Filipe de Avila Belbute Peres, Michael Petrov, Henrique Ponde de Oliveira Pinto, Michael, Pokorny, Michelle Pokrass, Vitchyr H. Pong, Tolly Powell, Alethea Power, Boris Power, Elizabeth Proehl, Raul Puri, Alec Radford, Jack Rae, Aditya Ramesh, Cameron Raymond, Francis Real, Kendra Rimbach, Carl Ross, Bob Rotsted, Henri Roussez, Nick Ryder, Mario Saltarelli, Ted Sanders, Shibani Santurkar, Girish Sastry, Heather Schmidt, David Schnurr, John Schulman, Daniel Selsam, Kyla Sheppard, Toki Sherbakov, Jessica Shieh, Sarah Shoker, Pranav Shyam, Szymon Sidor, Eric Sigler, Maddie Simens, Jordan Sitkin, Katarina Slama, Ian Sohl, Benjamin Sokolowsky, Yang Song, Natalie Staudacher, Felipe Petroski Such, Natalie Summers, Ilya Sutskever, Jie Tang, Nikolas Tezak, Madeleine B. Thompson, Phil Tillet, Amin Tootoonchian, Elizabeth Tseng, Preston Tuggle, Nick Turley, Jerry Tworek, Juan Felipe Cerón Uribe, Andrea Vallone, Arun Vijayvergiya, Chelsea Voss, Carroll Wainwright, Justin Jay Wang, Alvin Wang, Ben Wang, Jonathan Ward, Jason Wei, CJ Weinmann, Akila Welihinda, Peter Welinder, Jiayi Weng, Lilian Weng, Matt Wiethoff, Dave Willner, Clemens Winter, Samuel Wolrich, Hannah Wong, Lauren Workman, Sherwin Wu, Jeff Wu, Michael Wu, Kai Xiao, Tao Xu, Sarah Yoo, Kevin Yu, Qiming Yuan, Wojciech Zaremba, Rowan Zellers, Chong Zhang, Marvin Zhang, Shengjia Zhao, Tianhao Zheng, Juntang Zhuang, William Zhuk, and Barret Zoph. 2024. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL] https://arxiv.org/abs/2303.08774

[27] Desislava Paneva-Marinova, Lubomir Zlatkov, and Lilia Pavlova. 2019. Improved User Experience in Digital Library through Advanced Content Synthesizing. In *Proceedings of the 9th International Conference on Information Communication and Management* (Prague, Czech Republic) *(ICICM '19)*. Association for Computing Machinery, New York, NY, USA, 170–174. doi:10.1145/3357419.3357432

[28] Bo Qiao, Oliver Reiche, Frank Hannig, and Jürgen Teich. 2018. Automatic Kernel Fusion for Image Processing DSLs. In *Proceedings of the 21st International Workshop on Software and Compilers for Embedded Systems* (Sankt Goar, Germany) *(SCOPES '18)*. Association for Computing Machinery, New York, NY, USA, 76–85. doi:10.1145/3207719.3207723

[29] Bo Qiao, Oliver Reiche, Frank Hannig, and Jürgen Teich. 2019. From loop fusion to kernel fusion: a domain-specific approach to locality optimization. In *Proceedings of the 2019 IEEE/ACM International Symposium on Code Generation and Optimization* (Washington, DC, USA) *(CGO 2019)*. IEEE Press, 242–253.

[30] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2023. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. arXiv:1910.10683 [cs.LG] https://arxiv.org/abs/1910.10683

[31] Scott Reed, Konrad Zolna, Emilio Parisotto, Sergio Gomez Colmenarejo, Alexander Novikov, Gabriel Barth-Maron, Mai Gimenez, Yury Sulsky, Jackie Kay, Jost Tobias Springenberg, Tom Eccles, Jake Bruce, Ali Razavi, Ashley Edwards, Nicolas Heess, Yutian Chen, Raia Hadsell, Oriol Vinyals, Mahyar Bordbar, and Nando de Freitas. 2022. A Generalist Agent. arXiv:2205.06175 [cs.AI] https://arxiv.org/abs/2205.06175

[32] Amit Sabne. 2020. XLA : Compiling Machine Learning for Peak Performance.

[33] Chameleon Team. 2025. Chameleon: Mixed-Modal Early-Fusion Foundation Models. arXiv:2405.09818 [cs.CL] https://arxiv.org/abs/2405.09818

[34] Kimi Team, Yifan Bai, Yiping Bao, Guanduo Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, Zhuofu Chen, Jialei Cui, Hao Ding, Mengnan Dong, Angang Du, Chenzhuang Du, Dikang Du, Yulun Du, Yu Fan, Yichen Feng, Kelin Fu, Bofei Gao, Hongcheng Gao, Peizhong Gao, Tong Gao, Xinran Gu, Longyu Guan, Haiqing Guo, Jianhang Guo, Hao Hu, Xiaoru Hao, Tianhong He, Weiran He, Wenyang He, Chao Hong, Yangyang Hu, Zhenxing Hu, Weixiao Huang, Zhiqi Huang, Zihao Huang, Tao Jiang, Zhejun Jiang, Xinyi Jin, Yongsheng Kang, Guokun Lai, Cheng Li, Fang Li, Haoyang Li, Ming Li, Wentao Li, Yanhao Li, Yiwei Li, Zhaowei Li, Zheming Li, Hongzhan Lin, Xiaohan Lin, Zongyu Lin, Chengyin Liu, Chenyu Liu, Hongzhang Liu, Jingyuan Liu, Junqi Liu, Liang Liu, Shaowei Liu, T. Y. Liu, Tianwei Liu, Weizhou Liu, Yangyang Liu, Yibo Liu, Yiping Liu, Yue Liu, Zhengying Liu, Enzhe Lu, Lijun Lu, Shengling Ma, Xinyu Ma, Yingwei Ma, Shaoguang Mao, Jie Mei, Xin Men, Yibo Miao, Siyuan Pan, Yebo Peng, Ruoyu Qin, Bowen Qu, Zeyu Shang, Lidong Shi, Shengyuan Shi, Feifan Song, Jianlin Su, Zhengyuan Su, Xinjie Sun, Flood Sung, Heyi Tang, Jiawen Tao, Qifeng Teng, Chensi Wang, Dinglu Wang, Feng Wang, Haiming Wang, Jianzhou Wang, Jiaxing Wang, Jinhong Wang, Shengjie Wang, Shuyi Wang, Yao Wang, Yejie Wang, Yiqin Wang, Yuxin Wang, Yuzhi Wang, Zhaoji Wang, Zhengtao Wang, Zhexu Wang, Chu Wei, Qianqian Wei, Wenhao Wu, Xingzhe Wu, Yuxin Wu, Chenjun Xiao, Xiaotong Xie, Weimin Xiong, Boyu Xu, Jing Xu, Jinjing Xu, L. H. Xu, Lin Xu, Suting Xu, Weixin Xu, Xinran Xu, Yangchuan Xu, Ziyao Xu, Junjie Yan, Yuzi Yan, Xiaofei Yang, Ying Yang, Zhen Yang, Zhilin Yang, Zonghan Yang, Haotian Yao, Xingcheng Yao, Wenjie Ye, Zhuorui Ye, Bohong Yin, Longhui Yu, Enming Yuan, Hongbang Yuan, Mengjie Yuan, Haobing Zhan, Dehao Zhang, Hao Zhang, Wanlu Zhang, Xiaobin Zhang, Yangkun Zhang, Yizhi Zhang, Yongting Zhang, Yu Zhang, Yutao Zhang, Yutong Zhang, Zheng Zhang, Haotian Zhao, Yikai Zhao, Huabin Zheng, Shaojie Zheng, Jianren Zhou, Xinyu Zhou, Zaida Zhou, Zhen Zhu, Weiyu Zhuang, and Xinxing Zu. 2025. Kimi K2: Open Agentic Intelligence. arXiv:2507.20534 [cs.LG] https://arxiv.org/abs/2507.20534

[35] Vijay Thakkar, Pradeep Ramani, Cris Cecka, Aniket Shivam, Honghao Lu, Ethan Yan, Jack Kosaian, Mark Hoemmen, Haicheng Wu, Andrew Kerr, Matt Nicely, Duane Merrill, Dustyn Blasig, Fengqi Qiao, Piotr Majcher, Paul Springer, Markus Hohnerbach, Jin Wang, and Manish Gupta. 2023. *CUTLASS*. https://github.com/NVIDIA/cutlass

[36] Philippe Tillet, Hsiang-Tsung Kung, and David Cox. 2019. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*. 10–19.

[37] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).

[38] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).

[39] Mohamed Wahib and Naoya Maruyama. 2014. Scalable kernel fusion for memory-bound GPU applications. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (New Orleans, Louisana) *(SC '14)*. IEEE Press, 191–202. doi:10.1109/SC.2014.21

[40] Lei Wang, Yu Cheng, Yining Shi, Zhengju Tang, Zhiwen Mo, Wenhao Xie, Lingxiao Ma, Yuqing Xia, Jilong Xue, Fan Yang, et al. 2025. TileLang: A Composable Tiled Programming Model for AI Systems. *arXiv preprint arXiv:2504.17577* (2025).

[41] Chunwei Xia, Jiacheng Zhao, Qianqi Sun, Zheng Wang, Yuan Wen, Teng Yu, Xiaobing Feng, and Huimin Cui. 2024. Optimizing deep learning inference via global analysis and tensor expressions. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 286–301.

[42] Jiarong Xing, Leyuan Wang, Shang Zhang, Jack Chen, Ang Chen, and Yibo Zhu. 2022. Bolt: Bridging the gap between auto-tuners and hardware-native performance. *Proceedings of Machine Learning and Systems* 4 (2022), 204–216.

[43] Lei Xu, Zhiwen Mo, Qin Wang, Jianfei Jiang, and Naifeng Jing. 2024. Enabling Multiple Tensor-wise Operator Fusion for Transformer Models on Spatial Accelerators. In *Proceedings of the 61st ACM/IEEE Design Automation Conference* (San Francisco, CA, USA) *(DAC '24)*. Association for Computing Machinery, New York, NY, USA, Article 232, 6 pages. doi:10.1145/3649329.3657317

[44] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505. 09388* (2025).

[45] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, et al. 2025. Flashinfer: Efficient and customizable attention engine for llm inference serving. *arXiv preprint arXiv:2501.01005* (2025).

[46] Zheng Zhang, Donglin Yang, Xiaobo Zhou, and Dazhao Cheng. 2024. MCFuser: High-performance and rapid fusion of memory-bound compute-intensive operators. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–15.

[47] Size Zheng, Siyuan Chen, Peidi Song, Renze Chen, Xiuhong Li, Shengen Yan, Dahua Lin, Jingwen Leng, and Yun Liang. 2023. Chimera: An analytical optimizing framework for effective compute-intensive operators fusion. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 1113–1126.

[48] Yaowei Zheng, Richong Zhang, Suyuchen Wang, Samuel Mensah, and Yongyi Mao. 2020. Anchored Model Transfer and Soft Instance Transfer for Cross-Task Cross-Domain Learning: A Study Through Aspect-Level Sentiment Classification. In *Proceedings of The Web Conference 2020* (Taipei, Taiwan) *(WWW '20)*. Association for Computing Machinery, New York, NY, USA, 2754–2760. doi:10.1145/3366423.3380034

[49] Zhen Zheng, Xuanda Yang, Pengzhan Zhao, Guoping Long, Kai Zhu, Feiwen Zhu, Wenyi Zhao, Xiaoyong Liu, Jun Yang, Jidong Zhai, et al. 2022. AStitch: enabling a new multi-dimensional optimization space for memory-intensive ML training and inference on modern SIMT architectures. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 359–373.

A Appendix

A.1 Non-invertible Condition

In the derivation presented in Chapter 3.2.2, we assumed that the function $\mathrm{H}_i(\hat{\mathbf{d}}^{k-1})$ is invertible within $(\mathbb{S}, \otimes_i)$, to support the existence of the inverse term $\mathrm{H}_i(\hat{\mathbf{d}}^{k-1})^{-1}$ in the fused expression. However, in certain practical scenarios (e.g., when $\otimes_i = *$ and $\mathrm{H}_i(\cdot) = 0$), this inverse may not exist, rendering the expression undefined.

To ensure semantic correctness of the fusion transformation in all cases, we propose a **reversibility repair mechanism**: extend $\mathrm{H}_i(\hat{\mathbf{d}}^{k-1})$ into a corrected function $\mathrm{H}'_i(\hat{\mathbf{d}}^{k-1})$, defined as:

$$\mathrm{H}'_i(\hat{\mathbf{d}}^{k-1}) = \begin{cases} \mathrm{H}_i(\hat{\mathbf{d}}^{k-1}), & \text{if it is invertible in } (\mathbb{S}, \otimes_i), \\ e, & \text{otherwise,} \end{cases} \tag{27}$$

where e is the identity element of $(\mathbb{S}, \otimes_i)$ (e.g. 0 for $(\mathbb{R}, +)$, 1 for $(\mathbb{R}, *)$). Since e is always invertible ($e^{-1} = e$), this extension guarantees that $\mathrm{H}'_i(\hat{\mathbf{d}}^{k-1})$ is invertible for all inputs.

Substituting H'_i into the Equation 11:

$$\hat{d}^k = \sum_{j^{k-1}=l^k_{1st}}^{l^k_{last}\,\sharp_i} \hat{d}^{k-1} \otimes_i \mathrm{H}'_i(\hat{\mathbf{d}}^{k-1})^{-1} \otimes_i \mathrm{H}'_i(\hat{\mathbf{d}}^{k}). \tag{28}$$

This corrected expression is computable in all cases and is equivalent to the original expression whenever $\mathrm{H}_i(\hat{\mathbf{d}}^{k-1})$ is invertible. Thus, it ensures robustness and correctness of the fused reduction computation across all input configurations.

A.2 Cases of Fused and Incremental Computation Expressions

A.2.1 Attention Mechanism. For a given token corresponding to a query vector, let $P = \frac{QK^T}{\sqrt{d_k}}$. The output of the attention computation can be expressed as:

$$\begin{aligned} m &= \max_{l=1}^{L_0} P[l], \\ t &= \sum_{l=1}^{L_0} \exp(P[l] - m), \\ O &= \sum_{l=1}^{L_0} \frac{\exp(P[l] - m)}{t} * V[l], \end{aligned} \tag{29}$$

where $V[l]$ denotes the l-th row of the value matrix V.

The fused computation form for the first level reduction at the j^1-th segment is

$$\begin{aligned} \hat{m}^1 &= \max_{l=l^1_{1st}}^{l^1_{last}} P[l], \\ \hat{t}^1 &= \sum_{l=l^1_{1st}}^{l^1_{last}} \exp\left(P[l] - \hat{m}^1\right), \\ \hat{O}^1 &= \sum_{l=l^1_{1st}}^{l^1_{last}} \frac{\exp\left(P[l] - \hat{m}^1\right)}{\hat{t}^1}. \end{aligned} \tag{30}$$

For level-k:

$$\begin{aligned} \hat{m}^k &= \max_{j^{k-1}=l^k_{1st}}^{l^k_{last}} \hat{m}^{k-1}, \\ \hat{t}^k &= \sum_{j^{k-1}=l^k_{1st}}^{l^k_{last}} \hat{t}^{k-1} * \exp\left(\hat{m}^{k-1} - \hat{m}^k\right), \\ \hat{O}^k &= \sum_{j^{k-1}=l^k_{1st}}^{l^k_{last}} \hat{O}^{k-1} * \exp\left(\hat{m}^{k-1} - \hat{m}^k\right) * \frac{\hat{t}^{k-1}}{\hat{t}^k}. \end{aligned} \tag{31}$$

The incremental computation form for the first level on the j^1-th input segment is:

$$\begin{aligned} \hat{m}^1[L] &= \max\left(\hat{m}^1[L-1],\, P[L]\right), \\ \hat{t}^1[L] &= \hat{t}^1[L-1] * \exp\left(\hat{m}^1[L-1] - \hat{m}^1[L]\right) \\ &\quad + \exp\left(P[L] - \hat{m}^1[L]\right), \\ \hat{O}^1[L] &= \hat{O}^1[L-1] * \exp\left(\hat{m}^1[L-1] - \hat{m}^1[L]\right) \\ &\quad * \frac{\hat{t}^1[L-1]}{\hat{t}^1[L]} + \frac{\exp\left(P[L] - \hat{m}^1[L]\right)}{\hat{t}^1[L]}. \end{aligned} \tag{32}$$

For level-k:

$$\begin{aligned} \hat{m}^k[L] &= \max\left(\hat{m}^k[L-1],\, \hat{m}^{k-1}\right), \\ \hat{t}^k[L] &= \hat{t}^k[L-1] * \exp\left(\hat{m}^k[L-1] - \hat{m}^k[L]\right) \\ &\quad + \hat{t}^{k-1} * \exp\left(\hat{m}^{k-1} - \hat{m}^k[L]\right), \\ \hat{O}^k[L] &= \hat{O}^k[L-1] * \exp\left(\hat{m}^k[L-1] - \hat{m}^k[L]\right) * \frac{\hat{t}^k[L-1]}{\hat{t}^k[L]} \\ &\quad + \hat{O}^{k-1} * \exp\left(\hat{m}^{k-1} - \hat{m}^k[L]\right) * \frac{\hat{t}^{k-1}}{\hat{t}^k[L]}. \end{aligned} \tag{33}$$

Notably, Equation (33) is identical to the tiled attention computation formula proposed in FlashAttention, which corresponds to fusion at level $k = 3$.

A.2.2 MoE routing. Since softmax preserves the relative ordering of input values, we partition the expert scores into segments of length K' for input to the TopK operation. The full computation is:

$$\begin{aligned} m &= \max_{l=1}^{L_0} x[l], \\ t &= \sum_{l=1}^{L_0} \exp(x[l] - m), \\ \vec{s} &= \mathrm{TopK}_{l=1}^{L_0/K'} \vec{x}[l]. \end{aligned} \tag{34}$$

where $\vec{s}$ and $\vec{x}$ are vectors of length K'. The fused expression at level k is defined as follows. For $k = 1$:

$$\begin{aligned}\hat{m}^1 &= \max_{l=l^1_{1st}}^{l^1_{last}} x[l],\\ \hat{t}^1 &= \sum_{l=l^1_{1st}}^{l^1_{last}} \exp\left(x[l] - \hat{m}^1\right),\\ \hat{\vec{s}}^1 &= \mathrm{TopK}_{l=l^1_{1st}}^{l^1_{last}} \vec{x}[l].\end{aligned} \tag{35}$$

For $k > 1$:

$$\begin{aligned}\hat{m}^k &= \max_{j^{k-1}=l^k_{1st}}^{l^k_{last}} \hat{m}^{k-1}, k > 1\\ \hat{t}^k &= \sum_{j^{k-1}=l^k_{1st}}^{l^k_{last}} \hat{t}^{k-1} * \exp\left(\hat{m}^{k-1} - \hat{m}^k\right),\\ \hat{\vec{s}}^k &= \mathrm{TopK}_{j^{k-1}=l^k_{1st}}^{l^k_{last}} \hat{\vec{s}}^{k-1}.\end{aligned} \tag{36}$$

The incremental form at level $k = 1$ is:

$$\begin{aligned}\hat{m}^1[L] &= \max\left(\hat{m}^1[L-1],\, x[l]\right),\\ \hat{t}^1[L] &= \hat{t}^1[L-1] * \exp\left(\hat{m}^1[L-1] - \hat{m}^1[L]\right)\\ &\quad + \exp\left(x[L] - \hat{m}^1[L]\right),\\ \hat{\vec{s}}^1[L] &= \mathrm{TopK}(\hat{\vec{s}}^1[L-1], \vec{x}[L]).\end{aligned} \tag{37}$$

For $k > 1$:

$$\begin{aligned}\hat{m}^k[L] &= \max\left(\hat{m}^k[L-1],\, \hat{m}^{k-1}\right),\\ \hat{t}^k[L] &= \hat{t}^k[L-1] * \exp\left(\hat{m}^k[L-1] - \hat{m}^k[L]\right)\\ &\quad + \hat{t}^{k-1} * \exp\left(\hat{m}^{k-1} - \hat{m}^k[L]\right),\\ \hat{\vec{s}}^k[L] &= \mathrm{TopK}(\hat{\vec{s}}^k[L-1], \hat{\vec{s}}^{k-1}).\end{aligned} \tag{38}$$

The final output can be obtained by normalizing the selected top-K' values: $\vec{s}/t$

A.2.3 Sum + Sum. This is a pattern used within the internal model, and its computational formulation is as follows:

$$\begin{aligned}m &= \sum_{l=1}^{L_0} x_1[l]^2,\\ s &= \sum_{l=1}^{L_0} \frac{x_1[l] * x_2[l]}{\sqrt{\max\left(m - 10\right)}}.\end{aligned} \tag{39}$$

The fused expression at level k is defined as follows. For $k = 1$:

$$\begin{aligned}\hat{m}^1 &= \sum_{l=l^1_{1st}}^{l^1_{last}} x_1[l]^2,\\ \hat{s}^1 &= \sum_{l=l^1_{1st}}^{l^1_{last}} \frac{x_1[l] * x_2[l]}{\sqrt{\max\left(\hat{m}^1 - 10\right)}}.\end{aligned} \tag{40}$$

For $k > 1$:

$$\begin{aligned}\hat{m}^k &= \sum_{j^{k-1}=l^k_{1st}}^{l^k_{last}} \hat{m}^{k-1},\\ \hat{s}^k &= \sum_{j^{k-1}=l^k_{1st}}^{l^k_{last}} \hat{s}^{k-1} * \sqrt{\frac{\max\left(\hat{m}^{k-1} - 10\right)}{\max\left(\hat{m}^k - 10\right)}}.\end{aligned} \tag{41}$$

The incremental form at level $k = 1$ is:

$$\begin{aligned}\hat{m}^1[L] &= \hat{m}^1[L-1] + x_1[L]^2,\\ \hat{s}^1[L] &= \hat{s}^1[L-1] * \sqrt{\frac{\max\left(\hat{m}^1[L-1] - 10\right)}{\max\left(\hat{m}^1[L] - 10\right)}}\\ &\quad + \frac{x_1[L] * x_2[L]}{\sqrt{\max\left(\hat{m}^1[L] - 10\right)}}.\end{aligned} \tag{42}$$

For $k > 1$:

$$\begin{aligned}\hat{m}^k[L] &= \hat{m}^k[L-1] + \hat{m}^{k-1},\\ \hat{s}^k[L] &= \hat{s}^k[L-1] * \sqrt{\frac{\max\left(\hat{m}^k[L-1] - 10\right)}{\max\left(\hat{m}^k[L] - 10\right)}}\\ &\quad + \hat{s}^{k-1} * \sqrt{\frac{\max\left(\hat{m}^{k-1} - 10\right)}{\max\left(\hat{m}^k[L] - 10\right)}}.\end{aligned} \tag{43}$$

A.3 TileOp Specification

We formalize the TileOps used in RedFuser in Figure 10, with semantics as follows:

- `copy(src, dst)`: Copies data from tile `src` to `dst`.
- `gemm(A, B, C)`: Computes `C += A * B`.
- `reduce(src, dst, axis = k, op)`: Reduces tile `src` along dimension `k` using binary operator `op`, storing the result in `dst`.
- `parallel(buf[idx+], f(args*), iters+, ranges+)`: Evaluates `buf[idx+] = f(args*)` in parallel over the iteration space defined by `iters+` and `ranges+`.
- `fill(tile, c)`: Fills `tile` with constant `c`.

A.4 Implementation of FlashAttention and FlashDecoding

This section details the implementation of FlashAttention and FlashDecoding in RedFuser.

Beginning with the unfused TIR representation in Figure 11, RedFuser analyzes the AST to both identify four distinct reduction operations and determine their respective axes. Crucially, it detects that reductions 2-4 share a common axis, forming a cascaded reduction pattern. RedFuser then lifts this structure into equivalent mathematical expressions to facilitate fusion.

Using the ACRF algorithm (Section 4.2), RedFuser first obtains the fused expressions. These expressions are then realized into two scalar-level IR structures by applying the **Single-Segment** and **Multi-Segment** strategies (Section 4.3),

following a three-step reduction template: (1) store prior results, (2) correct current results, and (3) execute the reduction. Notably, RedFuser further eliminates redundant steps based on dataflow dependencies; for instance, step (2) is omitted for reductions with no dependencies (such as reduction 2), and step (1) is skipped for reductions whose outputs are not reused (such as reduction 4). The resulting scalar-level IR structures are shown in Figures 12a and 13a.

The scalar-level IR is subsequently elevated to tile-level IR via **Tensorization** and **Parallelization** (Section 4.4), and further optimized by TileLang into high-performance GPU kernels. We configure a tile size of (`blk_qs`, `blk_kvs`) = (128, 128) for both FlashAttention and FlashDecoding, and set `num_split = 2` for FlashDecoding. The final tile-level IR structures are illustrated in Figures 12b and 13b.

A.5 Workload Configurations

We present in detail the configurations of all selected subgraphs.

A.5.1 MHA and MLA. For MHA, the input tensors are shaped as $[bs, hn, q, hd]$ for query and $[bs, hn, kv, hd]$ for key and value, where bs, hn, q, kv and hd denote for *batch size*, *head num*, *sequence length q*, *sequence length kv* and *head dim*. In MLA, we set the query length to always be 1 to simulate the decoding stage of LLM. The hidden dimensions of query and key are extended by the RoPE embedding dimension (ped). Detailed configurations are shown in Table 2a and Table 2b.

A.5.2 MoE routing and Quant GEMM. In MoE routing, the expert scores are computed via a GEMM between an input tensor of shape $[s, hd]$ and a routing weight matrix of shape $[hd, en]$, where s, hd and en denote for *sequence length*, *head dim* and *expert num*. For Quant+GEMM, we consider a dynamic quantization workflow where a matrix of shape $[M, K]$ is quantized to FP8 using a reduction-based scaling factor, followed by a GEMM with a weight matrix of shape $[K, N]$. Table 2c and Table 2d provide detailed configurations.

A.6 Evaluation on non-ML Workloads

For non-ML workloads, we evaluate two representative examples: (1) variance computation, and (2) moment of inertia computation in 3D space relative to the center of mass. The formula for variance is as follows:

$$\begin{aligned} m &= \frac{1}{L_0}\sum_{l=1}^{L_0} x[l], \\ var &= \frac{1}{L_0}\sum_{l=1}^{L_0}(x[l]-m)^2. \end{aligned} \tag{44}$$

The moment of inertia about the center of mass is computed as:

$$\begin{aligned} M &= \sum_{l=1}^{L_0} m[l], \\ \vec{c} &= \frac{1}{M}\sum_{l=1}^{L_0} m[l] * \vec{x[l]}, \\ I &= \sum_{l=1}^{L_0} m[l] * \|\vec{x[l]} - \vec{c}\|^2. \end{aligned} \tag{45}$$

As both formulas involve sequences of inter-dependent reductions, they exhibit cascaded reduction structures. Hence, they are amenable to fusion optimization within the RedFuser framework.

The configurations for non-ML workloads are summarized in Table 3. For variance computation, the input x is a 2D tensor of shape $[bs, l]$ where bs denotes the *batch size* and l denotes the *number of data points* in each batch. For moment of inertia computation, the mass input m is a 2D tensor of shape $[bs, n]$, and the position input x is a 3D tensor of shape $[bs, n, dim]$, where bs is the *batch size*, n is the *number of particles*, and dim (set to 3) represents the *spatial dimensions of 3D coordinates*.

The results are shown in Figure 8. For variance computation, since variance frequently appears in ML workloads (e.g., in various normalization operators), PyTorch includes optimized implementations. Nevertheless, RedFuser achieves an average speedup of 4.8×, 4.0×, 2.9× and 4.6× over PyTorch Eager on A10, A100, H800 and MI308X, respectively. For moment of inertia computation, the presence of extensive element-wise operations between reductions prevents existing compilers from effectively fusing the reduction chains, resulting in suboptimal performance. In this case, RedFuser achieves an average speedup of 6.4×, 5.5×, 6.2× and 11.6× on A10, A100, H800 and MI308X, respectively.

A.7 Evaluation Across Multiple Platforms

We evaluate RedFuser on the four workloads from Section 5.2 across NVIDIA A100, H800, and AMD MI308X GPUs. Compared to PyTorch Eager, RedFuser delivers average speedups of 6.7×, 3.9×, and 1.6× on MoE routing, and 6.9×, 7.9×, and 3.4× on MHA workloads, respectively. On FP8 per-token quantization with GEMM, it achieves a 2.0× speedup on MI308X.

Table 2. The configurations of four workloads

Name	bs	hn	q	kv	hd	model
H1	32	8	512	512	64	BERT-Small
H2	32	12	512	512	64	BERT-Base
H3	32	16	512	512	64	BERT-Large
H4	32	12	256	256	64	ViT-Base
H5	32	16	256	256	64	ViT-Large
H6	32	16	256	256	80	ViT-Huge
H7	32	64	1	1024	128	LLaMA-65B
H8	32	64	1	2048	128	LLaMA-65B
H9	32	64	1	4096	128	LLaMA-65B

(a) The configurations of MHA

Name	bs	hn	kv	hd	ped
L1	32	128	1024	512	64
L2	32	128	2048	512	64
L3	32	128	4096	512	64
L4	16	128	1024	512	64
L5	16	128	2048	512	64
L6	16	128	4096	512	64
L7	1	128	1024	512	64
L8	1	128	2048	512	64
L9	1	128	4096	512	64

(b) The configurations of MLA

Name	s	hd	en	topk	model
R1	2048	768	128	1	switch-base-128
R2	2048	1024	128	1	switch-large-128
R3	2048	4096	128	1	switch-xxl-128
R4	2048	2560	64	6	ERNIE-21B-A3B
R5	2048	8192	64	8	ERNIE-300B-A47B
R6	2048	2048	64	6	DeepSeek-V2-Lite
R7	2048	2048	128	8	Qwen3-30B-A3B
R8	2048	4096	128	8	Qwen3-235B-A30B

(c) The configurations of MoE routing

Name	M	N	K	model
Q1	4096	1536	2560	ERNIE-21B-A3B
Q2	4096	2560	1536	ERNIE-21B-A3B
Q3	4096	3584	8192	ERNIE-300B-A47B
Q4	4096	8192	3584	ERNIE-300B-A47B
Q5	4096	7168	2048	DeepSeek-R1
Q6	4096	2048	7168	DeepSeek-R1
Q7	4096	2048	768	Qwen3-30B-A3B
Q8	4096	768	2048	Qwen3-30B-A3B
Q9	4096	4096	1536	Qwen3-235B-A30B
Q10	4096	1536	4096	Qwen3-235B-A30B

(d) The configurations of Quant + GEMM

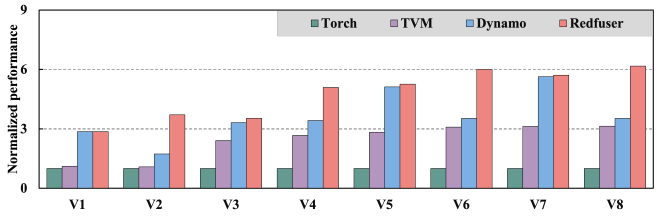
Table 3. The configurations of two non-ML workloads

Name	bs	l
V1	1	8192
V2	1	32768
V3	128	8192
V4	128	32768
V5	512	8192
V6	512	32768
V7	1024	8192
V8	1024	32768

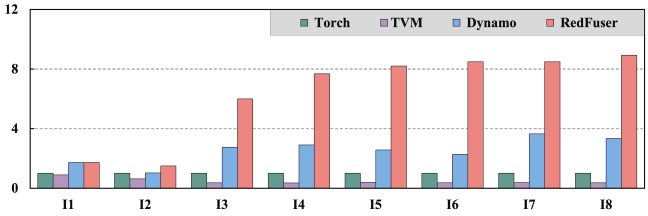
(a) The configurations of Variance

Name	bs	n	dim
I1	1	8192	3
I2	1	32768	3
I3	128	8192	3
I4	128	32768	3
I5	512	8192	3
I6	512	32768	3
I7	1024	8192	3
I8	1024	32768	3

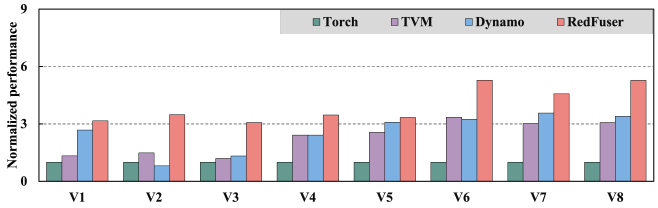
(b) The configurations of Moment of Inertia



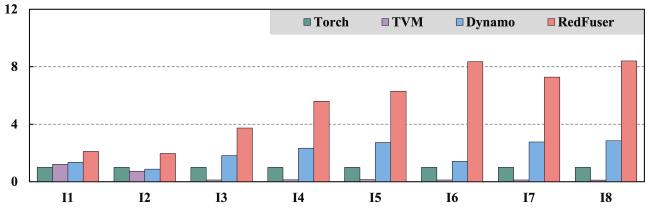
(a) Variance on A10



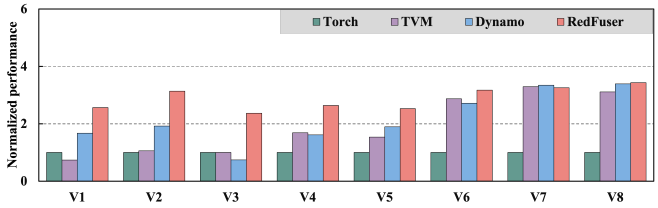
(b) Moment of inertia on A10



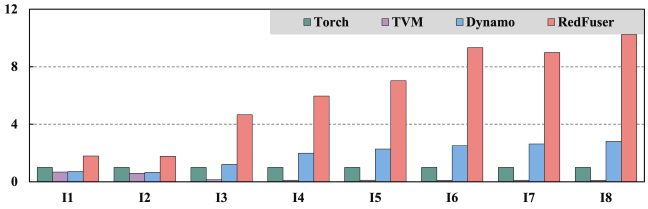
(c) Variance on A100



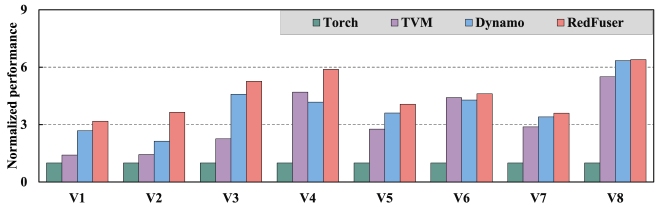
(d) Moment of inertia on A100



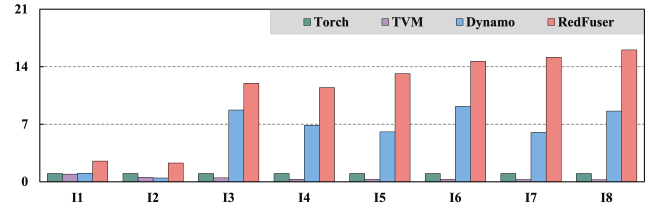
(e) Variance on H800



(f) Moment of inertia on H800

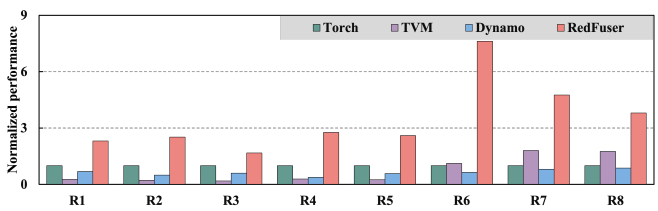


(g) Variance on MI308X

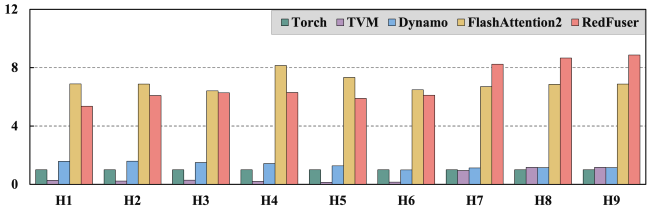


(h) Moment of Inertia on MI308X

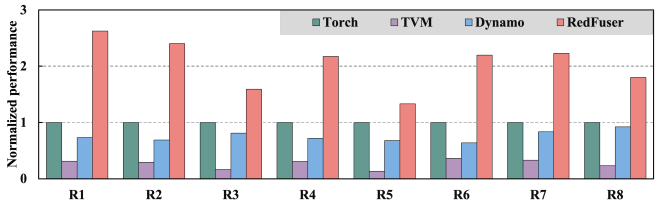
Figure 8. The normalized performance of non-ML workloads across multiple platforms.



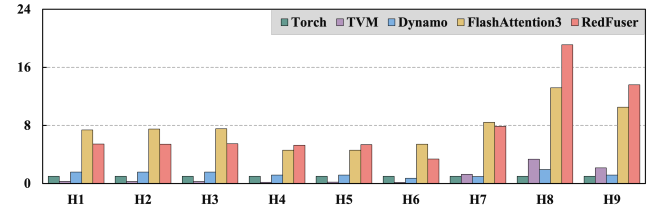
(a) MoE routing on A100



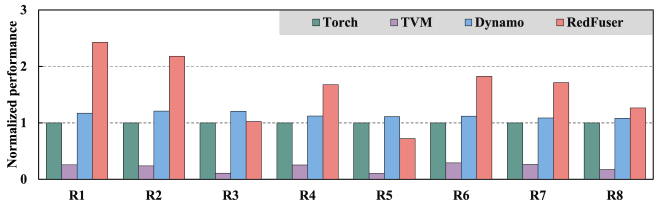
(b) MHA on A100



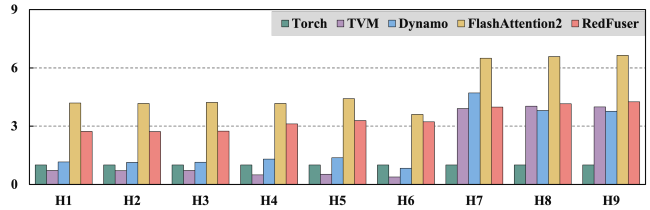
(c) MoE routing on H800



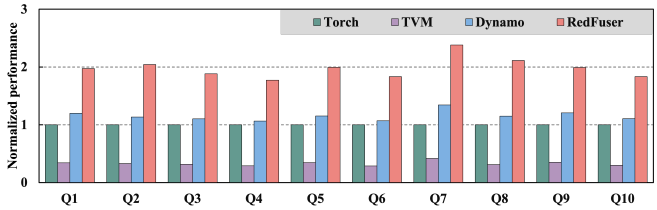
(d) MHA on H800



(e) MoE routing on MI308X



(f) MHA on MI308X



(g) FP8 PerToken Quant + GEMM on MI308X

Figure 9. The normalized performance of ML workloads across multiple platforms.

$$
\begin{aligned}
\mathit{TileOp} ::= & \ \texttt{copy}(\mathit{tile},\ \mathit{tile}) \\
& \mid \texttt{gemm}(\mathit{tile},\ \mathit{tile},\ \mathit{tile}) \\
& \mid \texttt{reduce}(\mathit{tile},\ \mathit{tile},\ \texttt{axis} = \mathit{lit},\ \mathit{op}) \\
& \mid \texttt{parallel}(\mathit{id}[\mathit{expr}+],\ \mathit{op}(\mathit{expr}*),\ \mathit{id}+,\ \mathit{range}+) \\
& \mid \texttt{fill}(\mathit{tile},\ \mathit{lit}) \\
\mathit{tile} ::= & \ \mathit{id}[\mathit{range}+] \\
\mathit{range} ::= & \ \mathit{expr} : \mathit{expr} \\
\mathit{expr} ::= & \ \mathit{lit} \mid \mathit{id} \mid \mathit{op}(\mathit{expr}*) \\
\mathit{op} ::= & \ + \mid - \mid \texttt{max} \mid \texttt{exp} \mid \ldots
\end{aligned}
$$

Figure 10. TileOp Specification

```
for qs in range(512):
    # reduction1: gemm(Q,K)
    for kvs in range(512):
        for d in range(64):
            P[qs, kvs] += Q[qs, d] * K[kvs, d]
    for kvs in range(512):  # reduction2: max(P)
        pmax[qs] = max(pmax[qs], P[qs, kvs])
    # reduction3: sum(exp(P - pmax))
    for kvs in range(512):
        psum[qs] += exp(P[qs, kvs] - pmax[qs])
    # reduction4: gemm(exp(P - pmax) / psum, V)
    for kvs in range(512):
        for d in range(64):
            o[qs, d] += exp(P[qs, kvs] - pmax[qs]) / psum
                [qs] * V[kvs, d]
```

Figure 11. Unfused attention TIR

```
for qs in range(512):
    for kvs in range(512):
        # reduction1: gemm(Q,K)
        for d in range(64):
            P[qs, kvs] += Q[qs, d] * K[kvs, d]

        # reduction2: max(P)
        # step 1. store previous result
        pmax_prev[qs] = pmax[qs]
        # step 3. perform reduction
        pmax[qs] = max(pmax[qs], P[qs, kvs])

        # reduction3: sum(exp(P - pmax))
        # step 1. store previous result
        psum_prev[qs] = psum[qs]
        # step 2. apply correction
        psum[qs] *= exp(pmax_prev[qs] - pmax[qs])
        # step 3. perform reduction
        psum[qs] += exp(P[qs, kvs] - pmax[qs])

        # reduction4: gemm(exp(P - pmax) / psum, V)
        # step 2. apply correction
        for d in range(64):
            o[qs, d] *= exp(pmax_prev[qs] - pmax[qs]) * (psum_prev[qs] / psum[qs])
        # step 3. perform reduction
        for d in range(64):
            o[qs, d] += exp(P[qs, kvs] - pmax[qs]) / psum[qs] * V[kvs, d]
```

(a) FlashAttention scalar-level IR

```
bx = launch_thread("blockIdx.x", 4)
# we omit the alloc shared / fragment buffer for brevity
copy(Q[bx * 128:bx * 128 + 128, 0:64], Q_shared[0:128, 0:64])
for stage in range(4):
    copy(K[stage * 128:stage * 128 + 128, 0:64], K_shared[0:128, 0:64])
    copy(V[stage * 128:stage * 128 + 128, 0:64], V_shared[0:128, 0:64])
    # reduction1: gemm(Q,K)
    gemm(Q_shared, K_shared, P_frag)

    # reduction2: max(P)
    # step 1. store previous result
    copy(pmax, pmax_prev)
    # step 3. perform reduction
    reduce(P_frag, pmax, axis=1, op=max)

    # reduction3: sum(exp(P - pmax))
    # step 1. store previous result
    copy(psum, psum_prev)
    # step 2. apply correction
    parallel(psum[i], psum[i] * exp(pmax_prev[i] - pmax[i]), i, 128)
    # step 3. perform reduction
    parallel(pexp[i, j], exp(P_frag[i, j] - pmax[i]), i, 128, j, 128)
    reduce(pexp, psum, axis=1, op=add)

    # reduction4: gemm(exp(P - pmax) / psum, V)
    # step 2. apply correction
    parallel(o_frag[i, j], o_frag[i, j] * exp(pmax_prev[i] - pmax[i]) * (psum_prev[i] / psum[i]), i, 128, j, 64)
    # step 3. perform reduction
    parallel(oexp[i, j], exp(P_frag[i, j] - pmax[i]) / psum[i], i, 128, j, 128)
    gemm(oexp, V_shared, o_frag)
copy(o_frag[0:128, 0:64], o[bx * 128:bx * 128 + 128, 0:64])
```

(b) FlashAttention tile-level IR

Figure 12. FlashAttention IR

```
# partial segment
for qs, split in grid(512, 2):
    for kvs in range(256):
        offset = split * 256 + kvs
        for d in range(64):
            P[qs, offset] += Q[qs, d] * K[offset, d]
        pmax_part_prev[qs, split] = pmax_part[qs, split]
        pmax_part[qs, split] = max(pmax_part[qs, split], P[qs, offset])

        psum_part_prev[qs, split] = psum_part[qs, split]
        psum_part[qs, split] *= exp(pmax_part_prev[qs, split] - pmax_part[qs, split])
        psum_part[qs, split] += exp(P[qs, offset] - pmax_part[qs, split])

        for d in range(64):
            o_part[qs, d, split] *= exp(pmax_part_prev[qs, split] - pmax_part[qs, split]) * (psum_part_prev[qs, split]
                / psum_part[qs, split])
        for d in range(64):
            o_part[qs, d, split] += exp(P[qs, offset] - pmax_part[qs, split]) / psum_part[qs, split] * V[offset, d]
# combine
for qs in range(512):
    for split in range(2):
        pmax[qs] = max(pmax[qs], pmax_part[qs, split])
    for split in range(2):
        psum_part[qs, split] *= exp(pmax_part[qs, split] - pmax[qs])
        psum[qs] += psum_part[qs, split]
    for split in range(2):
        for d in range(64):
            o_part[qs, d, split] *= exp(pmax_part[qs, split] - pmax[qs]) * (psum_part[qs, split] / psum[qs])
        for d in range(64):
            o[qs, d] += o_part[qs, d, split]
```

(a) FlashDecoding scalar-level IR

```
# partial segment
bx = launch_thread("blockIdx.x", 4)
by = launch_thread("blockIdx.y", 2)
copy(Q[bx * 128:bx * 128 + 128, 0:64], Q_shared[0:128, 0:64])
for stage in range(2):
    offset = by * 256 + stage * 128
    copy(K[offset:offset + 128, 0:64], K_shared[0:128, 0:64])
    copy(V[offset:offset + 128, 0:64], V_shared[0:128, 0:64])
    gemm(Q_shared, K_shared, P_frag)
    copy(pmax_frag, pmax_frag_prev)
    reduce(P_frag, pmax_frag, axis=1, op=max)
    copy(psum_frag, psum_frag_prev)
    parallel(psum_frag[i], psum_frag[i] * exp(pmax_frag_prev[i] - pmax_frag[i]), i, 128)
    parallel(pexp[i, j], exp(P_frag[i, j] - pmax_frag[i]), i, 128, j, 128)
    reduce(pexp, psum_frag, axis=1, op=add)
    parallel(o_frag[i, j], o_frag[i, j] * exp(pmax_frag_prev[i] - pmax_frag[i]) * (psum_frag_prev[i] / psum_frag[i]), i
        , 128, j, 64)
    parallel(oexp[i, j], exp(P_frag[i, j] - pmax_frag[i]) / psum_frag[i], i, 128, j, 64)
    gemm(oexp, V_shared, o_frag)
copy(pmax_frag[0:128], pmax_part[bx * 128:bx * 128 + 128, by:by+1])
copy(psum_frag[0:128], psum_part[bx * 128:bx * 128 + 128, by:by+1])
copy(o_frag[0:128, 0:64], o_part[bx * 128:bx * 128 + 128, 0:64, by:by+1])
# combine
bx = launch_thread("blockIdx.x", 4)
copy(pmax_part[bx * 128:bx * 128 + 128, 0:2], pmax_frag[0:128, 0:2])
copy(psum_part[bx * 128:bx * 128 + 128, 0:2], psum_frag[0:128, 0:2])
copy(o_part[bx * 128:bx * 128 + 128, 0:64, 0:2], o_frag[0:128, 0:64, 0:2])
reduce(pmax_frag, pmax, axis=1, op=max)
parallel(psum_frag[i, j], psum_frag[i, j] * exp(pmax_frag[i, j] - pmax[i]), i, 128, j, 2)
reduce(psum_frag, psum, axis=1, op=add)
parallel(o_frag[i, j, k], o_frag[i, j, k] * exp(pmax_frag[i, k] - pmax[i]) * (psum_frag[i, k] / psum[i]), i, 128, j,
    64, k, 2)
reduce(o_frag, o_final, axis=2, op=add)
copy(o_final[0:128, 0:64], o[bx * 128:bx * 128 + 128, 0:64])
```

(b) FlashDecoding tile-level IR

Figure 13. FlashDecoding IR